# Computer Vision · Week 1

Beginner study guide

Day 1 – Images, pixels & colour spaces
Day 2 – Image transformations
Day 3 – Filtering, convolution & edge detection
Day 4 – Histograms & thresholding
Day 5 – Morphological operations
Day 6 – Contours & shape analysis
Key concepts – corners, SIFT, SURF, HOG, stereo, optical flow, Kalman, tracking

Python/NumPy basics · Notes · Diagrams · Course question bank answers · Practice quiz with answers · Cheat sheet
Runnable code is in the **code/** folder next to this PDF.

## 00 · Just enough Python & NumPy for Week 1

You don't need to be a programmer. The lectures use **three libraries**, and you only need a small piece of each.

| Library | Imported as | Job in plain words |
|---|---|---|
| OpenCV | `import cv2` | The image toolbox: load, convert colours, rotate, blur, find edges… |
| NumPy | `import numpy as np` | Stores grids of numbers (an image *is* one) and does maths on them |
| Matplotlib | `import matplotlib.pyplot as plt` | Draws/shows pictures and charts on screen |

---

## 1. A NumPy array = a grid of numbers

```
import numpy as np

a = np.array([[1, 2, 3],
              [4, 5, 6]])
print(a.shape)   # (2, 3)  -> 2 rows, 3 columns
print(a[0, 2])   # 3       -> row 0, column 2  (counting starts at 0!)
print(a[1])      # [4 5 6] -> the whole of row 1
```

- `.shape` tells you the size: `(rows, columns)`, or `(rows, columns, channels)` for colour images.
- **Counting starts at 0.** The first row is row 0.
- `a[row, col]`: the row always comes first.

## 2. Slicing = cutting out a piece

`start:stop` means "from start up to **but not including** stop".

```
img[100:200, 50:300]   # rows 100..199, columns 50..299  -> a crop
img[:, 0]              # ":" alone means "all" -> every row, column 0
img[0:60, 0:60] = 255  # set a whole block at once (paint it white)
```

## 3. Data types: `uint8` and `float64`

- Images are usually `uint8`: whole numbers from **0 to 255** only.
- Going past 255 **wraps around** in NumPy: `250 + 10 = 4` (!). OpenCV's `cv2.add` **clips** instead: `250 + 10 = 255`.
- Maths that can go negative (edge detection) needs `float64` (`cv2.CV_64F` in OpenCV).
- `np.float32([...])`: OpenCV's transformation functions want their matrices as 32-bit decimals. That's why the lectures write `np.float32([[1,0,100],[0,1,50]])`.

## 4. Useful NumPy bits seen in the lectures

```
np.zeros((3, 3))          # 3x3 grid of 0s
np.ones((3, 3)) / 9       # 3x3 grid of 1/9  (an "average" kernel)
np.array([[0,-1,0],[-1,5,-1],[0,-1,0]])  # the sharpening kernel
a * b                     # multiply element by element (same-size grids)
np.sum(a)                 # add everything up
```

## 5. Showing an image with Matplotlib

```
plt.imshow(img_rgb)            # colour image (must be RGB order!)
plt.imshow(gray, cmap='gray')  # 1-channel image: tell it to use grey shades
plt.title("My title")
plt.axis('off')                # hide the x/y ruler numbers
plt.show()                     # actually display it

plt.subplot(1, 2, 1)           # "1 row, 2 columns, draw in slot 1"
plt.subplot(1, 2, 2)           # "...now slot 2"  -> side-by-side pictures
```

> Without `cmap='gray'`, a 1-channel image is drawn in yellow-green-purple colours (matplotlib's default colour map). It's the same data, just painted with fake colours.

## 6. Python bits used in the mini-projects

```
def image_viewer(img_path, mode="original"):   # a function with a default value
    if mode == "gray":
        ...
    elif mode == "edges":       # "else if"
        ...

f"Mode: {mode}"                # f-string: puts the variable's value into the text

for i, cnt in enumerate(contours):   # loop that also gives the counter i = 0, 1,
2...
    ...

filters = {"blur": ..., "sharpen": ...}   # dictionary: name -> value
for name, output in filters.items():      # loop over (name, value) pairs
    ...

x = a if condition else b                 # one-line if/else
```

That's all the Python you need for Week 1. Next: `day1_images_and_colors.md`.

# Day 1 · Introduction to Computer Vision

**Lecture covers:** What is CV? · Images as data · Colour spaces · Transformations (overview) · Filtering (intro) · Applications · Mini-project: Image Viewer

**Code for today:** `code/day1_01` → `day1_04`

---

## 1. What is Computer Vision?

> **Computer Vision (CV)** is the part of AI that lets computers *see*: take in pictures or video and work out what's in them.

Your eyes just see a cat. A computer receives **millions of numbers** and has to work out from those numbers that there's a cat. CV is the set of tricks for doing that.

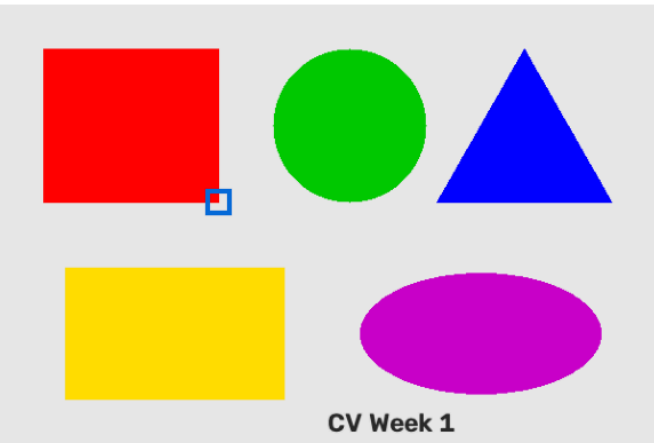
**Applications the lecture lists (learn these for the eval):**

| Area | Example |
|---|---|
| Self-driving cars | Detect lanes, cars, pedestrians, traffic signs |
| Medical imaging | Find tumours in X-rays/MRI scans |
| Surveillance | Spot people or unusual activity on CCTV |
| Augmented Reality (AR) | Snapchat filters, Pokémon GO (stick graphics onto the real world) |
| Traffic monitoring | Count / detect cars |
| Retail | Face recognition, product tracking on shelves |
| Creative AI | Art generation |

---

## 2. Images as data

What you see: a picture




What the computer sees: numbers (0 = black, 255 = white)
8×8 zoom of the blue box (corner of the red square), in grayscale

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 76 | 76 | 76 | 76 | 76 | 230 | 230 | 230 |
| 76 | 76 | 76 | 76 | 76 | 230 | 230 | 230 |
| 76 | 76 | 76 | 76 | 76 | 230 | 230 | 230 |
| 76 | 76 | 76 | 76 | 76 | 230 | 230 | 230 |
| 76 | 76 | 76 | 76 | 76 | 230 | 230 | 230 |
| 230 | 230 | 230 | 230 | 230 | 230 | 230 | 230 |
| 230 | 230 | 230 | 230 | 230 | 230 | 230 | 230 |
| 230 | 230 | 230 | 230 | 230 | 230 | 230 | 230 |

- A digital image is a **grid of tiny squares called pixels** (short for "picture elements").
- Each pixel holds **numbers that say how bright it is**.
- **Grayscale image:** 1 number per pixel. `0 = black`, `255 = white`, in between = grey.
- **Colour image:** 3 numbers per pixel, one each for **Red, Green, Blue**.
- Why 0–255? Each number is stored in **8 bits** (1 byte), and 8 bits can hold $2^8 = 256$ different values. The type is called **`uint8`** (unsigned 8-bit integer).
- **Resolution** = how many pixels the image has (width × height). More pixels means more detail. The lecture's photo was **5472 × 3648 ≈ 20 million pixels**.

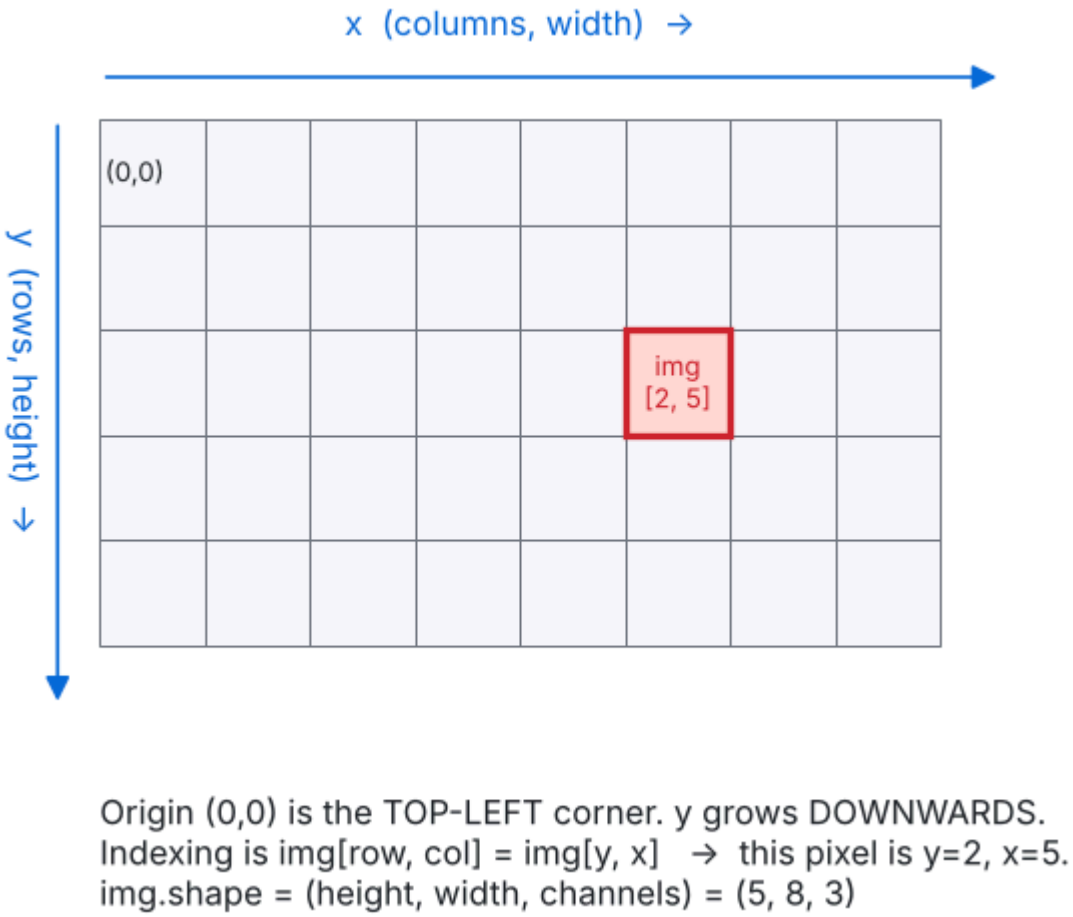
### Shape: the most important 3 numbers

```
img = cv2.imread('sample.jpg')
print(img.shape)        # (5472, 3648, 3)  in the lecture
#                          height  width  channels
```

| Position | Meaning | Other name |
|---|---|---|
| `shape[0]` | **height**, the number of rows | `rows` |
| `shape[1]` | **width**, the number of columns | `cols` |
| `shape[2]` | **channels** (3 for colour; missing for grayscale) | |

> ⚠️ Shape is **(height, width)**, but OpenCV functions that ask for a *size* want **(width, height)**. E.g. `cv2.warpAffine(img, M, (cols, rows))`. This flip catches everyone at least once.

### Coordinates: the origin is the TOP-LEFT




- `(0, 0)` is the **top-left** corner (not the bottom-left like in school maths graphs).
- **x** goes right (columns). **y** goes **down** (rows).
- To read a pixel: `img[y, x]`, i.e. **row first**.

```
print(img[50, 50])     # [172 169 161]  in the lecture → B=172, G=169, R=161 (a light grey)
```

Why grey? When all three numbers are roughly equal, the colour is grey. All 0 is black, all 255 is white.

### The BGR trap ⚠️ (very likely to be asked)

- OpenCV loads colour images in **B, G, R** order (Blue first). That's a historical choice.
- Matplotlib (and almost everything else) expects **R, G, B**.
- So you **must convert before showing with matplotlib**:

```
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(img_rgb)
```

- If you forget, **red and blue swap**. The lecture's *Mode: original* screenshot in the mini-project shows exactly that: the orange Pantone swatches came out **blue**, because the function passed the BGR image straight to `plt.imshow`.

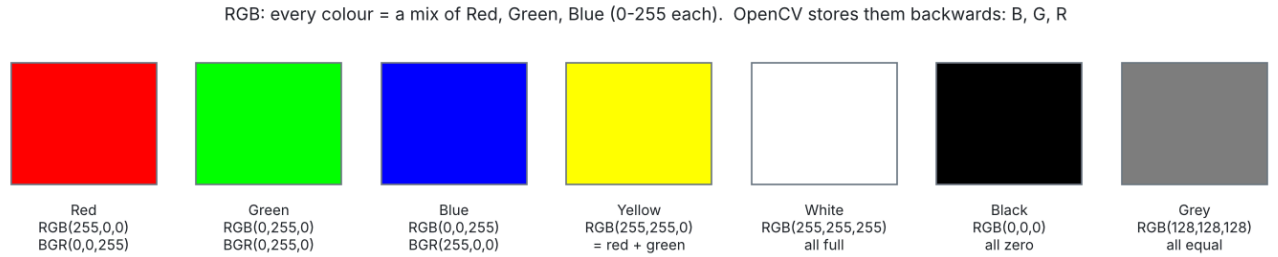
- `cv2.imshow()` (OpenCV's own window) expects BGR, so it needs **no** conversion.

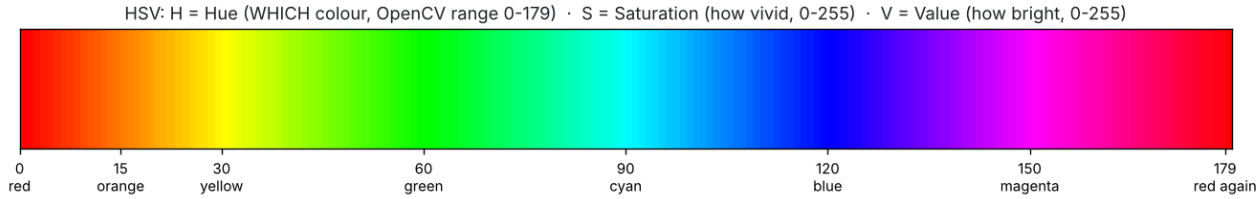
▶ Run `code/day1_01_load_and_inspect.py` and `code/day1_02_pixels_by_hand.py`.

---

## 3. Colour spaces

A **colour space** is just a *way of describing colour with numbers*. Same colour, different description.




### RGB (and OpenCV's BGR)

- Every colour = a mix of **Red + Green + Blue** light, each 0–255.
- Red + Green = Yellow. All full = White. All zero = Black.
- Problem: if a red ball goes into shadow, **all three** numbers change. That makes "find the red things" awkward.

### Grayscale

- One brightness number per pixel. 3× less data, and enough for edges and shapes.
- OpenCV's formula: **Gray = 0.299·R + 0.587·G + 0.114·B**. Green counts most because human eyes are most sensitive to green.
- `gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)` → shape becomes `(h, w)` with no channel number.

### HSV = Hue, Saturation, Value

Think of describing paint to a friend:

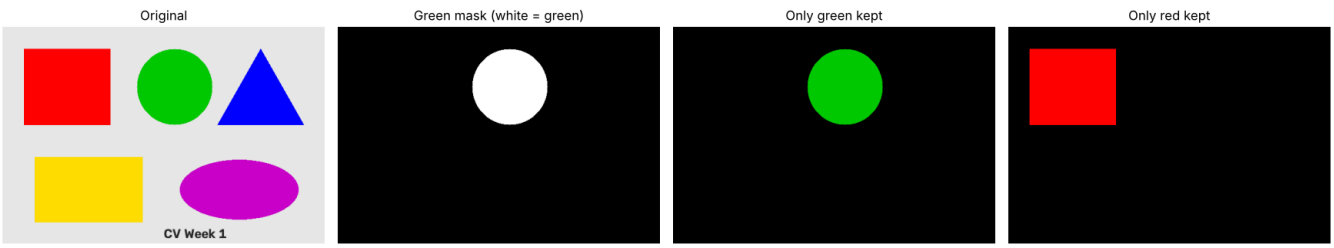
| Letter | Means | Plain words | OpenCV range |
|---|---|---|---|
| **H**ue | *Which* colour | "it's red / green / blue" (position on the colour wheel) | **0–179** (half of 0–360°, so it fits in a byte) |
| **S**aturation | How *pure / vivid* | 0 = washed-out grey, 255 = vivid | 0–255 |
| **V**alue | How *bright* | 0 = black, 255 = bright | 0–255 |

**Why HSV helps with colour detection:** the *colour* (Hue) is stored separately from the *brightness* (Value). A red ball in sunlight and in shadow keeps roughly the **same Hue**, so "find red" becomes "find Hue ≈ 0". That's why the lecture says converting between colour spaces "helps in tasks like colour detection".

```
hsv  = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
mask = cv2.inRange(hsv, (40, 80, 50), (80, 255, 255))   # white where the pixel is
green
only_green = cv2.bitwise_and(img, img, mask=mask)
```

- A **mask** is a black-and-white image: white (255) = "yes, keep this pixel", black (0) = "no".
- Red sits at **both ends** of the hue scale (0 and 179), so red needs two ranges joined with `|`.

**Lecture note:** the lecture converts to HSV and then *back* to RGB before showing it, so the "HSV Color Space" picture looks exactly like the original. To actually *see* HSV, show its H, S and V channels separately, as `day1_03_color_spaces.py` does.




▶ Run `code/day1_03_color_spaces.py`.

---

## 4. Transformations & filtering: Day 1 preview

Day 1 only *introduces* these. They're covered properly on Days 2 and 3.

- **Translation**: shifting the image. **Scaling**: resizing. **Rotation**: turning around a pivot. **Perspective**: changing the viewpoint, as if photographed from another angle.
- **Convolution** = sliding a small grid of numbers (a **kernel**) over the image. Filters made this way detect **edges, textures and patterns**, and this is the **foundation of CNNs** (Convolutional Neural Networks, the deep-learning models used for images).
- Day 1 already shows the sharpening kernel `[[0,-1,0],[-1,5,-1],[0,-1,0]]` and `cv2.Canny(img, 100, 200)` for edges.

---

## 5. Mini-project: Image Viewer

**Task:** load an image → convert to grayscale → apply edge detection → toggle between modes.

```
def image_viewer(img_path, mode="original"):
    img = cv2.imread(img_path)
    if mode == "gray":
        img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    elif mode == "edges":
        img = cv2.Canny(img, 100, 200)
    plt.imshow(img, cmap='gray' if mode != "original" else None)
    plt.title(f"Mode: {mode}")
    plt.axis('off')
    plt.show()
```

Read it line by line: - `mode="original"` is a **default**: if you don't pass a mode, you get "original". - Gray and edges results are **1-channel**, so they need `cmap='gray'`. - **Bug:** in "original" mode it shows a **BGR** image with matplotlib, which swaps red and blue. Fix: `img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` in an `else:` branch.

▶ Run `code/day1_04_mini_project_image_viewer.py`. It also has an interactive keyboard version for your own computer.

---

## ✅ Day 1: check yourself

1. What does `img.shape = (480, 640, 3)` mean? *(480 rows high, 640 columns wide, 3 colour channels)*
2. Which pixel is `img[10, 200]`? *(row 10 (y = 10), column 200 (x = 200))*
3. Why do colours look wrong in matplotlib after `cv2.imread`? *(OpenCV uses BGR, matplotlib expects RGB)*
4. Why is HSV better than RGB for detecting a coloured object? *(Hue holds the colour separately from the brightness, so shadows don't change it much)*
5. What range does Hue have in OpenCV? *(0–179)*

More questions: `practice/quiz.md`.

# Day 2 · Image Transformations

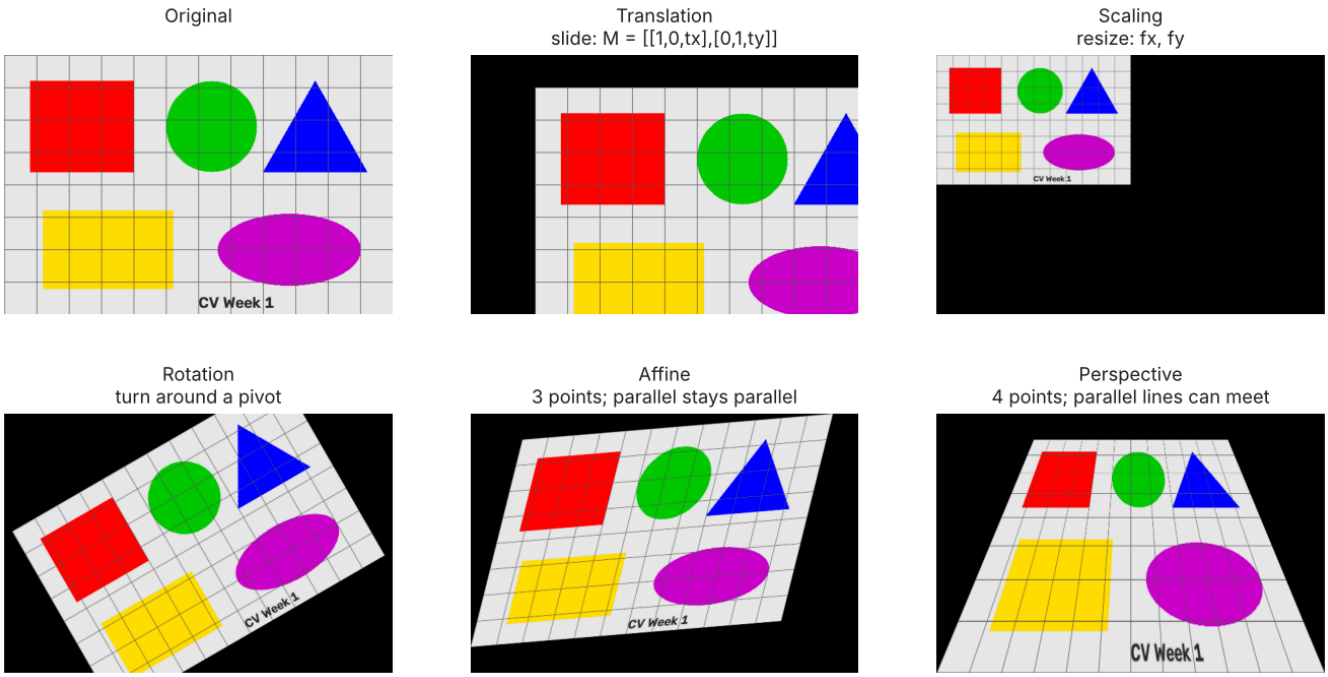
**Lecture covers:** Translation · Scaling (resizing) · Rotation · Affine transformations · Perspective transformations · Mini-project: Image Augmentation Tool

**Code for today:** `code/day2_01` → `day2_03`

---

## The big picture

A **transformation** moves pixels to **new positions**. The colours don't change, only *where* each pixel ends up.




Almost all of them follow the same 2-step recipe in OpenCV:

```
M   = <make a small matrix that describes the movement>
out = cv2.warpAffine(img, M, (width, height))      # or cv2.warpPerspective for 4-point warps
```

Think of **M** as a recipe card that says, for every pixel, "you started at (x, y), now go to (x', y')". `warpAffine` follows the recipe for every pixel.

⚠️ The lecture's Day 2 snippets use `rows` and `cols` without defining them first. You need:

```
rows, cols = img.shape[:2]     # rows = height, cols = width
```

---

## 1. Translation: sliding the picture

Shifting the image along x (right/left) and/or y (down/up).

```
M = np.float32([[1, 0, 100],     # x' = 1·x + 0·y + 100   → move 100 px RIGHT
                [0, 1,  50]])    # y' = 0·x + 1·y +  50   → move  50 px DOWN
translated = cv2.warpAffine(img, M, (cols, rows))
```

- General form: `[[1, 0, tx], [0, 1, ty]]`. `tx` is the horizontal shift, `ty` the vertical shift.
- **Negative** values move left/up.
- The area that becomes empty is filled with **black**, and anything pushed past the edge is cut off. That's why the lecture's "Translated Image" has a black strip on the top and left.
- The `1`s keep the size the same; the `0`s mean "x doesn't affect y and vice versa".

---

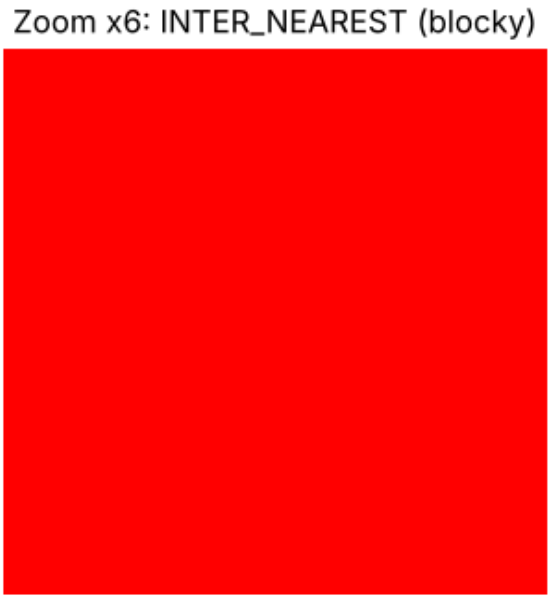
## 2. Scaling: resizing

```
scaled = cv2.resize(img, None, fx=0.5, fy=0.5, interpolation=cv2.INTER_LINEAR)
```

- `fx` = horizontal factor, `fy` = vertical factor. `0.5` = half, `2` = double.
- `None` = "I'm not giving an exact size, work it out from fx/fy". Or give the size directly: `cv2.resize(img, (300, 200))`. Remember that's **(width, height)**.
- Different fx and fy **stretch or squash** the image (the aspect ratio changes).

**Interpolation** = how to fill in pixels that didn't exist before (enlarging) or merge ones that get squeezed together (shrinking):

| Flag | How | Use it when |
|---|---|---|
| `INTER_NEAREST` | Copy the nearest pixel | Fastest, blocky look |
| `INTER_LINEAR` | Average of the 4 nearest | Default, good all-rounder |
| `INTER_CUBIC` | Uses 16 nearby pixels | Smoother when **enlarging** (slower) |
| `INTER_AREA` | Averages areas | Best for **shrinking** |







---

## 3. Rotation: turning around a pivot

```
M = cv2.getRotationMatrix2D((cols/2, rows/2), 45, 1)   # (centre, angle, scale)
rotated = cv2.warpAffine(img, M, (cols, rows))
```

| Argument | Meaning |
|---|---|
| `center` | The pivot point (x, y). `(cols/2, rows/2)` = the middle of the image |
| `angle` | Degrees. **Positive = anti-clockwise**, negative = clockwise |
| `scale` | `1` = same size. `0.6` = shrink while rotating, which keeps the corners inside |

- The output canvas stays the same size, so **the corners get cut off** and black triangles appear (see the lecture's "Rotated Image (45°)").
- The matrix OpenCV builds is `[[cos θ, sin θ, …], [−sin θ, cos θ, …]]`, scaled. For 45°, cos = sin ≈ 0.707.
- Shortcuts that don't crop: `cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)`. Mirror with `cv2.flip(img, 1)`.

---

## 4. Affine transformation: 3 points

> **Affine = keeps parallel lines parallel.** Straight lines stay straight, and lines that were parallel stay parallel. Squares can become slanted parallelograms, but never trapezoids.

Translation, scaling, rotation and **shear** (slanting, like italic text) are *all* affine. An affine transform is defined by **3 points before → 3 points after**:

```
pts1 = np.float32([[50, 50], [200, 50], [50, 200]])    # 3 points in the original
pts2 = np.float32([[10, 100], [200, 50], [100, 250]])  # where those 3 should go
M = cv2.getAffineTransform(pts1, pts2)                 # → 2×3 matrix
affine = cv2.warpAffine(img, M, (cols, rows))
```

Why 3? The affine matrix has **6 unknown numbers** (2×3). Each point pair gives 2 equations (one for x, one for y), so 3 pairs give the 6 equations needed.

---

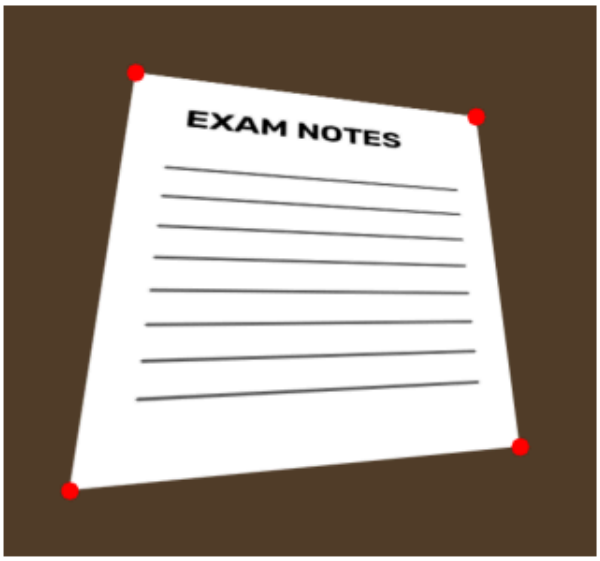
## 5. Perspective transformation: 4 points

> **Perspective maps any 4-sided shape to any other 4-sided shape.** Parallel lines can **meet**, like train tracks going into the distance. Straight lines still stay straight.

```
pts1 = np.float32([[56,65],[368,52],[28,387],[389,390]])   # 4 corners in the photo
pts2 = np.float32([[0,0],[300,0],[0,300],[300,300]])       # where they should go
M = cv2.getPerspectiveTransform(pts1, pts2)                # → 3×3 matrix
perspective = cv2.warpPerspective(img, M, (300, 300))
```

- Needs **4 point pairs** (the 3×3 matrix has 8 free numbers, and 4 pairs × 2 = 8 equations).
- Points must be listed in the **same order** in both lists (here: top-left, top-right, bottom-left, bottom-right).
- Uses `warpPerspective`, not `warpAffine`.
- **Main use, document scanning:** a page photographed at an angle is a tilted 4-sided shape. Map its 4 corners to a neat rectangle and you get a flat "scan". (Apps like CamScanner and Adobe Scan do this.)

Photo of paper (red = 4 corners)



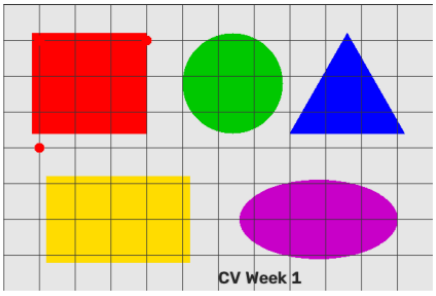

After warpPerspective = 'scanned'



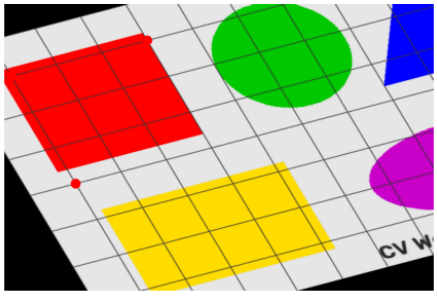

**Why the lecture's perspective output is a plain grey square:** its corner points (56,65)… (389,390) cover only a small patch in the top-left of a 5472×3648 photo, which is empty grey background. The code works, but the points weren't placed on an object. The 4 source points have to sit on the corners of the thing you want to flatten.

## Affine vs perspective (classic eval question)

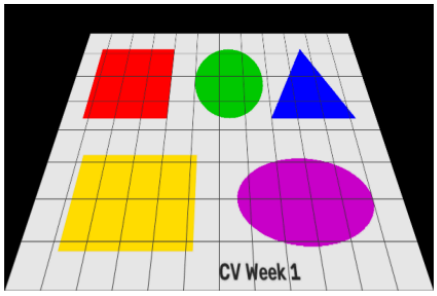
Original + grid (red = 3 source pts)




AFFINE: grid lines stay parallel



PERSPECTIVE: vertical lines now meet



| | Affine | Perspective |
|---|---|---|
| Points needed | **3** | **4** |
| Matrix size | **2×3** | **3×3** |
| Get matrix | `cv2.getAffineTransform` | `cv2.getPerspectiveTransform` |
| Apply | `cv2.warpAffine` | `cv2.warpPerspective` |
| Parallel lines | **stay parallel** | can **meet** |
| Rectangle can become | parallelogram | any 4-sided shape (e.g. trapezoid) |
| Includes | translate, scale, rotate, shear | all affine + "tilt / viewpoint change" |

▶ Run `code/day2_01_translate_scale_rotate.py` and `code/day2_02_affine_vs_perspective.py`.

---

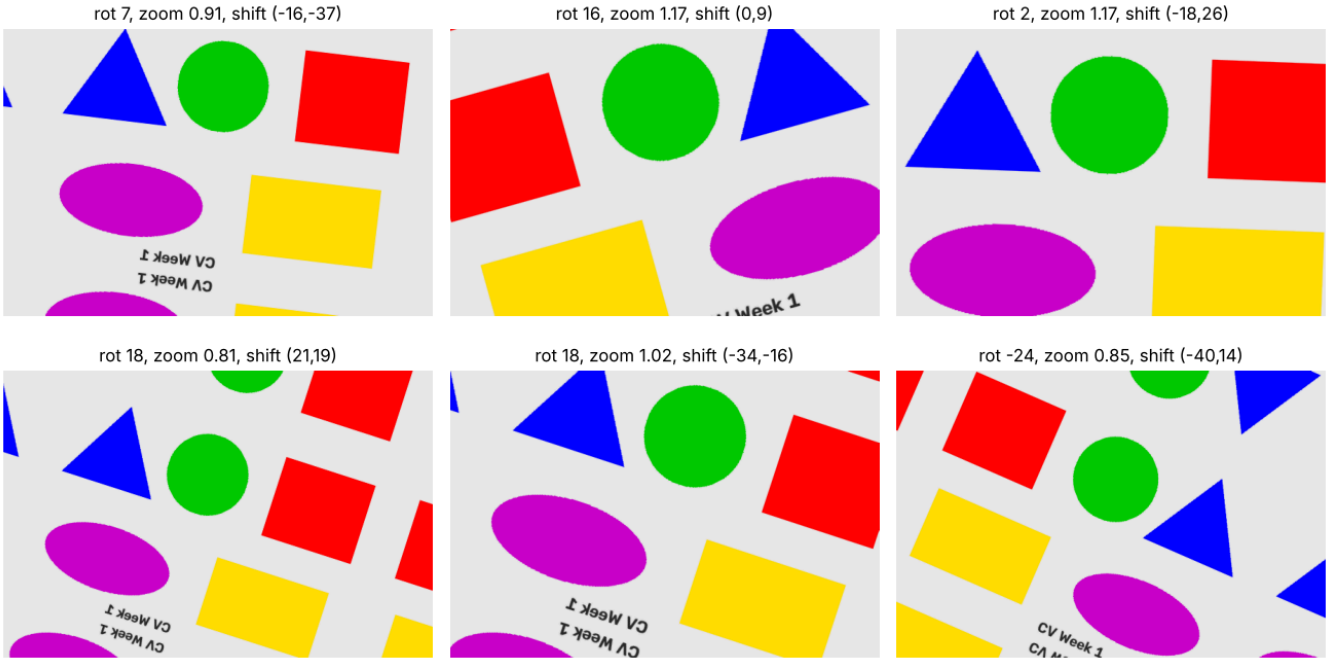
## 6. Mini-project: Image Augmentation Tool

**What is data augmentation?** AI models learn from examples, and you rarely have enough photos. So you create **extra versions** of each one: shifted, rotated, zoomed, flipped. It's still the same cat, so the model learns that a cat is a cat **whatever its position, size or angle**. More variety means the model generalises better and overfits less.

The lecture's `transform_image(img_path, mode)` uses `if / elif` to pick one transform:

| mode | does |
|---|---|
| `"translate"` | shift 100 right, 50 down |
| `"scale"` | resize to 50% |
| `"rotate"` | 45° around the centre |
| `"perspective"` | 4-point warp to 300×300 |

`day2_03_mini_project_augmentation.py` also has a **random augmenter** (random angle, zoom, shift and flip), which is how real deep-learning pipelines do it.




---

## ✅ Day 2: check yourself

1. Write the translation matrix for "move 30 px left and 20 px down". *( `[[1,0,-30], [0,1,20]` )*
2. What does the third argument of `getRotationMatrix2D` do? *(scale; 1 = same size)*
3. Is a positive angle clockwise or anti-clockwise? *(anti-clockwise)*
4. How many points do affine and perspective need? *(3 and 4)*
5. Which interpolation is best for shrinking? *( `INTER_AREA` )*
6. Why do corners disappear after rotation? *(the output canvas keeps the original size)*

# Day 3 · Filtering & Convolutions

**Lecture covers:** Why filtering matters · Convolution basics · Common filters (blur, sharpen, edge detection) · Canny edge detection · Applications · Lab exercise · Mini-project: Edge-based object detector

**Code for today:** `code/day3_01` → `day3_05`. **Start with** `day3_01_convolution_by_hand.py`: it's the key to the whole day.

---

## 1. Why filtering matters

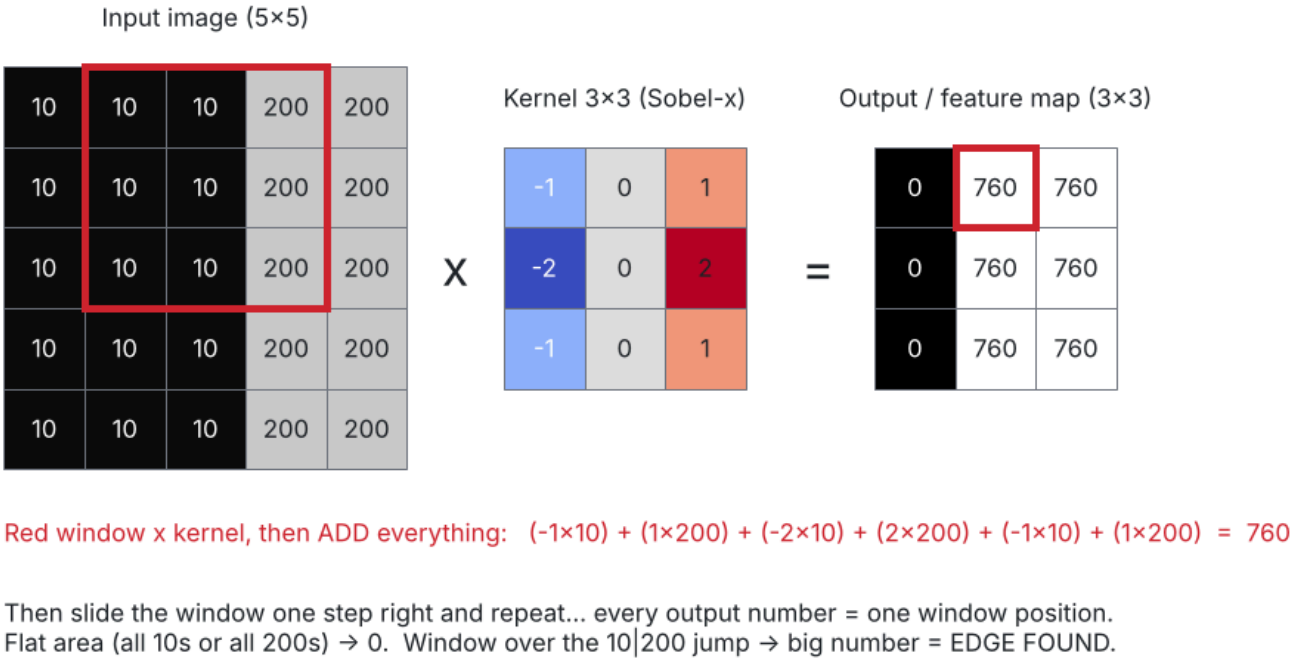
A **filter** changes each pixel based on **its neighbours**. Filters can:

- **Suppress** features: blur away noise or tiny details.
- **Enhance** features: sharpen, or highlight edges.
- They are the **foundation of CNNs**. A CNN is basically a stack of filters whose numbers the computer *learns by itself* instead of a human choosing them.

---

## 2. Convolution: the core idea

> **Kernel** = a small grid of numbers (usually 3×3 or 5×5). **Convolution** = slide the kernel over the image. At each spot, **multiply** each kernel number by the pixel under it, **add** them all up, and write that one number into the output. The output image is called a **feature map**: it highlights wherever the image matched the kernel's pattern.



Picture a magnifying glass with numbers written on it. You slide it across a photo one pixel at a time. At every spot you calculate one "score" for the patch underneath. Those scores form the new image.

**Worked example** (from the picture): window `[[10,10,200],[10,10,200],[10,10,200]]` × Sobel-x kernel `[[-1,0,1],[-2,0,2],[-1,0,1]]`:

```
(-1×10) + (0×10) + (1×200)
(-2×10) + (0×10) + (2×200)
(-1×10) + (0×10) + (1×200)   = -10+200 -20+400 -10+200 = 760   ← big number = EDGE
```

Over a flat area (all 10s): −10 + 10 − 20 + 20 − 10 + 10 = **0**, so no edge.

### The kernel-sum rule (easy marks)

| Kernel numbers add up to… | Effect | Examples |
|---|---|---|
| **1** | Overall brightness kept | Blur, sharpen, identity |
| **0** | Flat areas → 0 (black); only *changes* survive | Edge detectors (Sobel, Laplacian) |
| >1 / <1 | Image gets brighter / darker | |

### In OpenCV

```
kernel = np.array([[0,-1,0],[-1,5,-1],[0,-1,0]])
sharp  = cv2.filter2D(img, -1, kernel)     # -1 = output has the same depth/type as
the input
```

### Output size, padding, stride (CNN basics, often asked)

Without padding the kernel can't sit centred on border pixels, so the output **shrinks**: a 6×6 image with a 3×3 kernel gives 4×4. `cv2.filter2D` **pads** the border, so its output is the same size.

**Formula:** `output = (N − K + 2P) / S + 1` N = input size, K = kernel size, P = padding, S = stride (how many pixels the kernel jumps each step). Example: N = 6, K = 3, P = 0, S = 1 → (6 − 3 + 0)/1 + 1 = **4**.

**Fine print:** strictly speaking `filter2D` computes **correlation**. True convolution flips the kernel 180° first. For symmetric kernels (blur, sharpen) the results are identical. CNNs also use correlation and still call it "convolution".

▶ Run `code/day3_01_convolution_by_hand.py`. It does all of this with plain loops and proves it matches OpenCV.

---

## 3. Common filters

Same sliding-window maths, different numbers in the kernel → different effect

Identity (sum = 1)

| | | |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 0 | 0 | 0 |

Box blur (sum = 1)

| | | |
|---|---|---|
| 0.11 | 0.11 | 0.11 |
| 0.11 | 0.11 | 0.11 |
| 0.11 | 0.11 | 0.11 |

Sharpen (sum = 1)

| | | |
|---|---|---|
| 0 | -1 | 0 |
| -1 | 5 | -1 |
| 0 | -1 | 0 |

Sobel X (sum = 0)

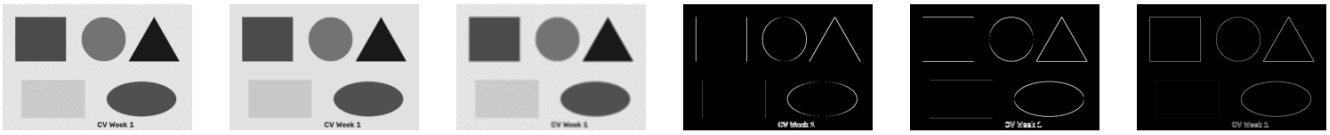
| | | |
|---|---|---|
| -1 | 0 | 1 |
| -2 | 0 | 2 |
| -1 | 0 | 1 |

Sobel Y (sum = 0)

| | | |
|---|---|---|
| -1 | -2 | -1 |
| 0 | 0 | 0 |
| 1 | 2 | 1 |

Laplacian (sum = 0)

| | | |
|---|---|---|
| 0 | 1 | 0 |
| 1 | -4 | 1 |
| 0 | 1 | 0 |



### Blur (smoothing): removes noise and fine detail

| Filter | Code | How it works |
|---|---|---|
| **Average (box)** | `cv2.blur(img, (5,5))` | Every pixel = plain average of its 5×5 neighbourhood. Kernel is all 1/25 |
| **Gaussian** | `cv2.GaussianBlur(img, (5,5), 0)` | Weighted average: **closer neighbours count more** (bell-curve weights). Looks more natural. `0` = let OpenCV choose sigma (spread) from the kernel size |
| Median *(bonus)* | `cv2.medianBlur(img, 5)` | Takes the middle value. Best for "salt-and-pepper" dots |

- The kernel size must be **odd** (3, 5, 7…) so there is a centre pixel.
- **Bigger kernel = more blur.**
- Blurring is often done **before** edge detection so that noise isn't mistaken for edges.

### Sharpen

```
kernel = np.array([[ 0, -1,  0],
                   [-1,  5, -1],
                   [ 0, -1,  0]])
```

"5 × me minus my 4 neighbours." In a flat area 5x − 4x = x, so nothing changes. At an edge the difference between neighbours gets **exaggerated**, so it looks crisper. The sum is 1, so brightness is kept.

### Edge detection: Sobel and Laplacian

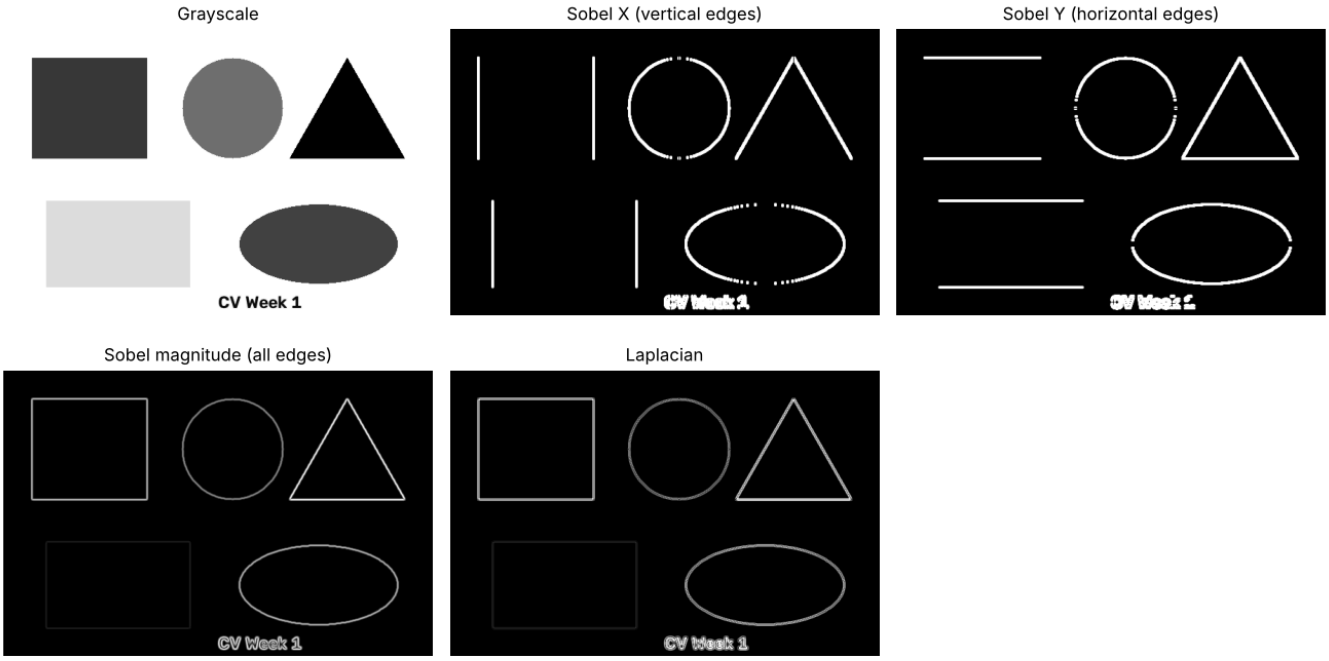
An **edge** is a place where brightness **changes suddenly**: the outline of an object.

**Sobel** measures how fast brightness changes in one direction (the *gradient*, i.e. the slope):

```
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=5)   # dx=1 → change left→right → VERTICAL edges
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=5)   # dy=1 → change top→bottom → HORIZONTAL edges
magnitude = np.sqrt(sobel_x**2 + sobel_y**2)           # all edges combined
```

- **Why `cv2.CV_64F`?** Dark→bright gives a positive value and bright→dark gives a **negative** one. `uint8` can't store negatives (they'd be cut to 0) and you'd lose half the edges. So compute in float, then `cv2.convertScaleAbs(...)` to take the absolute value and convert back to 0–255 for display.
- **The lecture's "Clipping input data to the valid range for imshow with RGB" warning:** it ran Sobel on the *colour* image and showed the float result (with negatives and values above 255) directly. That's why its Sobel picture looks rainbow-noisy. Fix: run Sobel on **grayscale** and use `convertScaleAbs`.

**Laplacian** looks at the *second* derivative (the change of the change) and finds edges in **all directions with one kernel** `[[0,1,0],[1,-4,1],[0,1,0]]`. It's very sensitive to noise, so blur first. `cv2.Laplacian(gray, cv2.CV_64F)`.




▶ Run `code/day3_02_common_filters.py`.

---

## 4. Canny edge detection

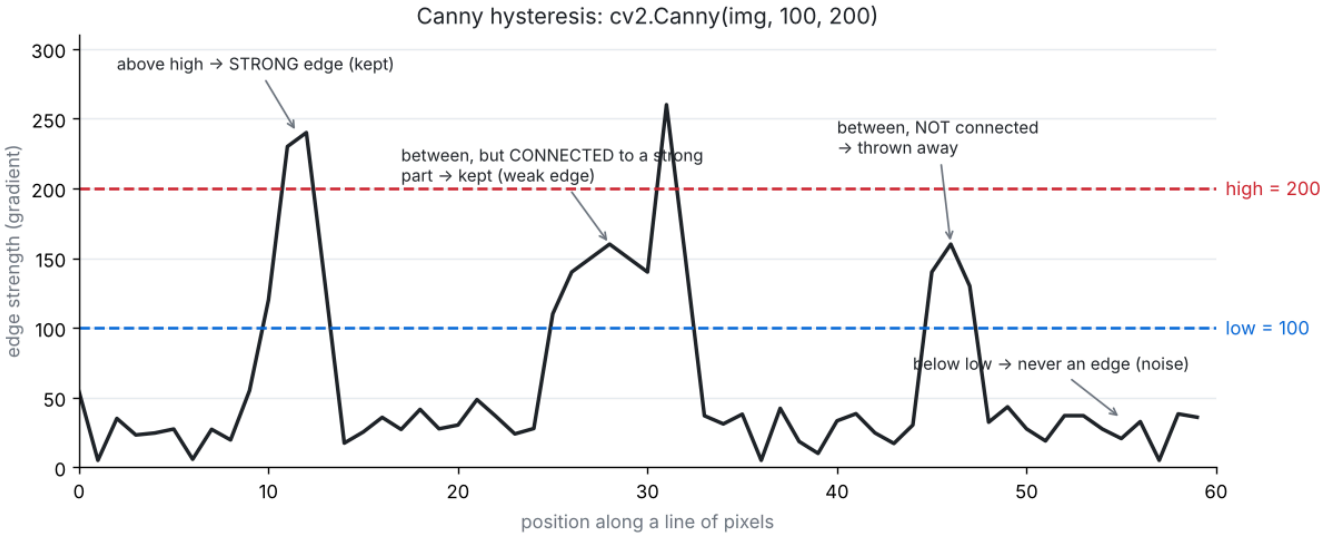
> "Combines gradient + thresholding. Produces clean edge maps."

The Canny algorithm has **4 steps**. Know them in order:

1. **Noise reduction**: Gaussian blur (5×5). *Fine print:* OpenCV's `cv2.Canny` function actually skips this step, which is why people usually call `cv2.GaussianBlur` themselves before

Canny. For the exam, list all 4 steps.

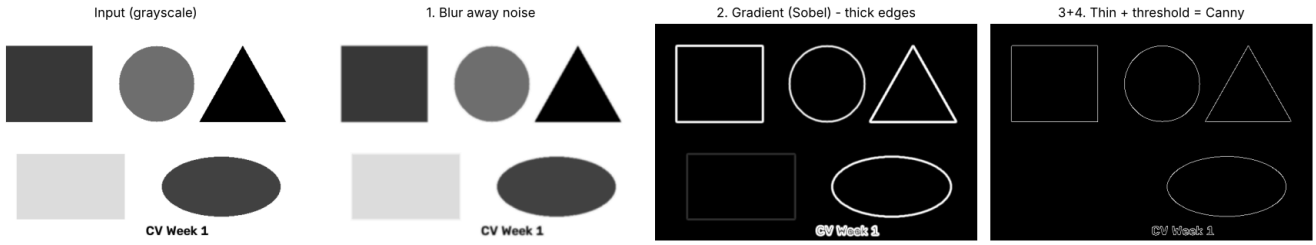
2. **Gradient**: Sobel in x and y gives the edge strength and direction at every pixel.
3. **Non-maximum suppression**: thick edges are thinned to **1 pixel wide** by keeping only the strongest pixel across each edge.
4. **Hysteresis thresholding** with the two numbers (low = 100, high = 200):




| Edge strength | Decision |
|---|---|
| **> high** (200) | **Strong edge**: keep |
| **< low** (100) | Not an edge: discard |
| **between** | Keep **only if connected to a strong edge**, otherwise discard |

This "connected" rule is what makes Canny's lines clean and continuous instead of broken and noisy.

- Output: a **single-channel, black image with white 1-pixel lines** (values only 0 or 255).
- **Lower thresholds** give more edges (including noise and texture). **Higher thresholds** keep only strong outlines. Rule of thumb: high ≈ 2–3 × low.




**Why the lecture's Canny images look almost black:** the photo is huge (5472×3648) and soft-focused. The edges are only 1 pixel thin, so when such a big image is shrunk to fit on the page most of them disappear. Resize the image first or lower the thresholds.

▶ Run `code/day3_03_canny.py`.

---

## 5. Applications of filtering

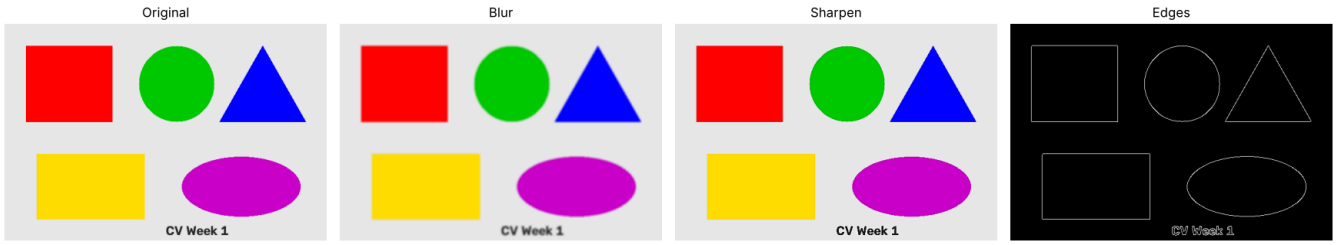
| Area | Use |
|---|---|
| Medical imaging | Find **tumour boundaries** (edges) |
| Document scanning | Find the page's edges and clean up/sharpen the text |
| Autonomous driving | **Lane detection** (edges of the lane markings) |

---

## 6. Lab exercise: compare filters side by side

```
filters = {
    "blur":    cv2.blur(img, (5,5)),
    "sharpen": cv2.filter2D(img, -1, kernel),
    "edges":   cv2.Canny(img, 50, 150),
}
for name, output in filters.items():
    plt.figure()
    plt.imshow(output if name=="edges" else cv2.cvtColor(output, cv2.COLOR_BGR2RGB),
               cmap='gray' if name=="edges" else None)
    plt.title(name.capitalize())
```

Why the `if name == "edges"`? Canny returns a **1-channel** image, which can't be converted BGR→RGB and needs `cmap='gray'`. Blur and sharpen return 3-channel BGR images, which need converting.




---

## 7. Mini-project: Edge-based object detector

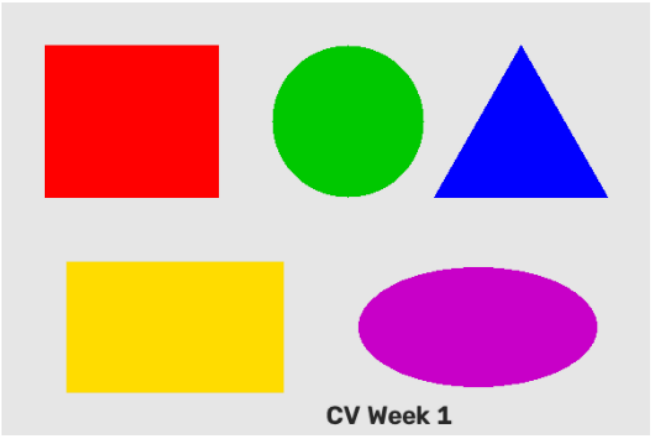
**Pipeline:** colour → grayscale → Canny edges → **find contours** → draw boxes and labels.

```
gray  = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 50, 150)
contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
for i, cnt in enumerate(contours):
    x, y, w, h = cv2.boundingRect(cnt)
    cv2.rectangle(img_contours, (x,y), (x+w,y+h), (0,255,0), 2)
    cv2.putText(img_contours, f"Obj{i+1}", (x,y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.5,
(0,255,0), 1)
```

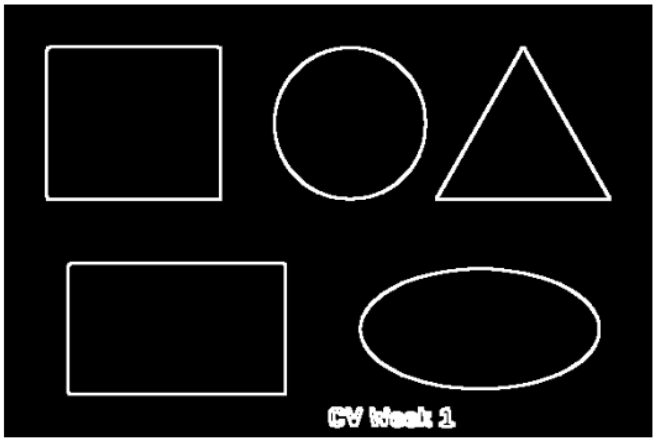
| Piece | Plain meaning |
|---|---|
| **Contour** | A list of points tracing the outline of one shape (like drawing around a coin with a pencil) |
| `findContours` returns | `(contours, hierarchy)`. Hierarchy says which outline is inside which. The lecture ignores it with `_` |
| `RETR_EXTERNAL` | Only the **outermost** outlines (ignore holes and inner shapes). `RETR_TREE` = all of them, with nesting |
| `CHAIN_APPROX_SIMPLE` | Store only the **corner points** of straight runs. A rectangle becomes 4 points instead of hundreds |
| `boundingRect(cnt)` | The smallest **upright box** around the contour: `x, y` = top-left corner, `w, h` = size |
| `rectangle(img, pt1, pt2, colour, thickness)` | Draws the box. `(0,255,0)` in BGR = **green** |
| `putText(img, text, position, font, scale, colour, thickness)` | Writes the label just above the box (`y-10`) |

**Why the lecture's result has green boxes all over the text and noise:** every tiny squiggle Canny finds becomes a "contour". Standard fixes (all in `day3_05_...py`): 1. **Blur first** (`GaussianBlur`) so there's less noise. 2. **Ignore small contours**: `if cv2.contourArea(cnt) < 500: continue`. 3. **Dilate** the edges to close tiny gaps so each shape is one closed outline. 4. *Bonus:* name each shape by counting corners with `cv2.approxPolyDP`: 3 = triangle, 4 = rectangle/square, many = circle.

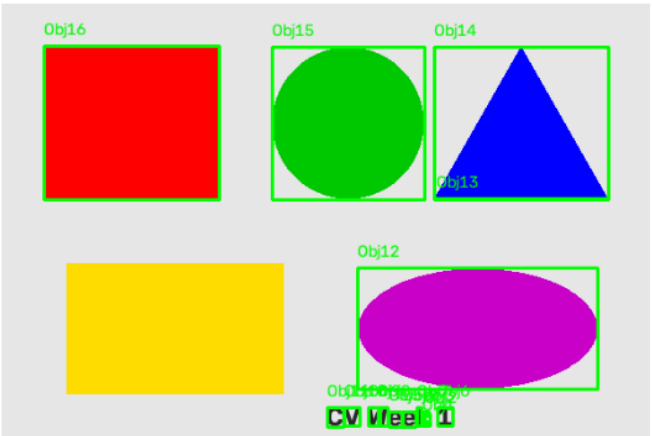
Input: simple shapes



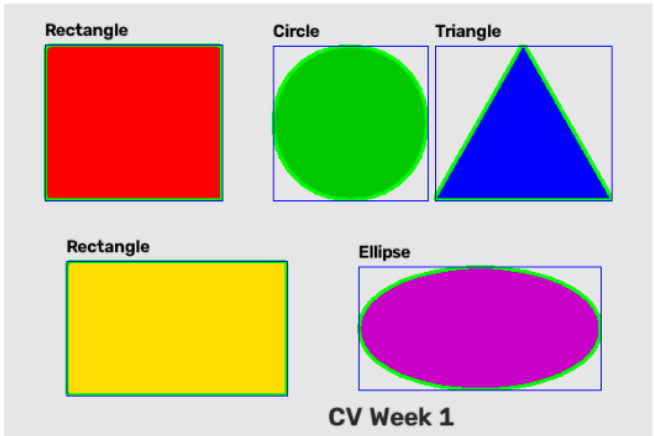

Edges (blur + Canny + dilate)



Lecture version: 16 'objects'




Improved: 5 labelled shapes



Notice the yellow rectangle: with the lecture's thresholds (50, 150) Canny **misses it**. Yellow (grey value 205) on light grey (230) is only 25 brightness levels apart. **Edges are found on brightness, not colour**, so two different colours with similar brightness look the same to an edge detector.

▶ Run `code/day3_05_mini_project_object_detector.py`.

---

## ✅ Day 3: check yourself

1. In one sentence, what is convolution? *(Slide a small kernel over the image and, at each spot, multiply and add to get one output pixel.)*
2. Sharpen kernel sum? Sobel kernel sum? What does each mean? *(1 keeps brightness; 0 makes flat areas black)*
3. Why `cv2.CV_64F` in Sobel? *(edge values can be negative)*
4. The 4 steps of Canny? *(blur → gradient → non-max suppression → hysteresis)*
5. In `Canny(img, 100, 200)`, what happens to a pixel with strength 150? *(kept only if connected to a strong edge, i.e. one above 200)*
6. Output size for a 32×32 input, 5×5 kernel, no padding, stride 1? *((32−5)/1+1 = 28 → 28×28)*
7. What does `RETR_EXTERNAL` do? *(returns only the outermost contours)*

# Day 4 · Histograms & Thresholding

**Lecture covers:** Image histograms · Global thresholding · Adaptive thresholding · Otsu's method · Applications · Lab exercise · Mini-project: Document Binarizer

**Code for today:** `code/day4_01` → `day4_03`

---

## 0. Loading straight to grayscale

From Day 4 on, the lectures load images like this:

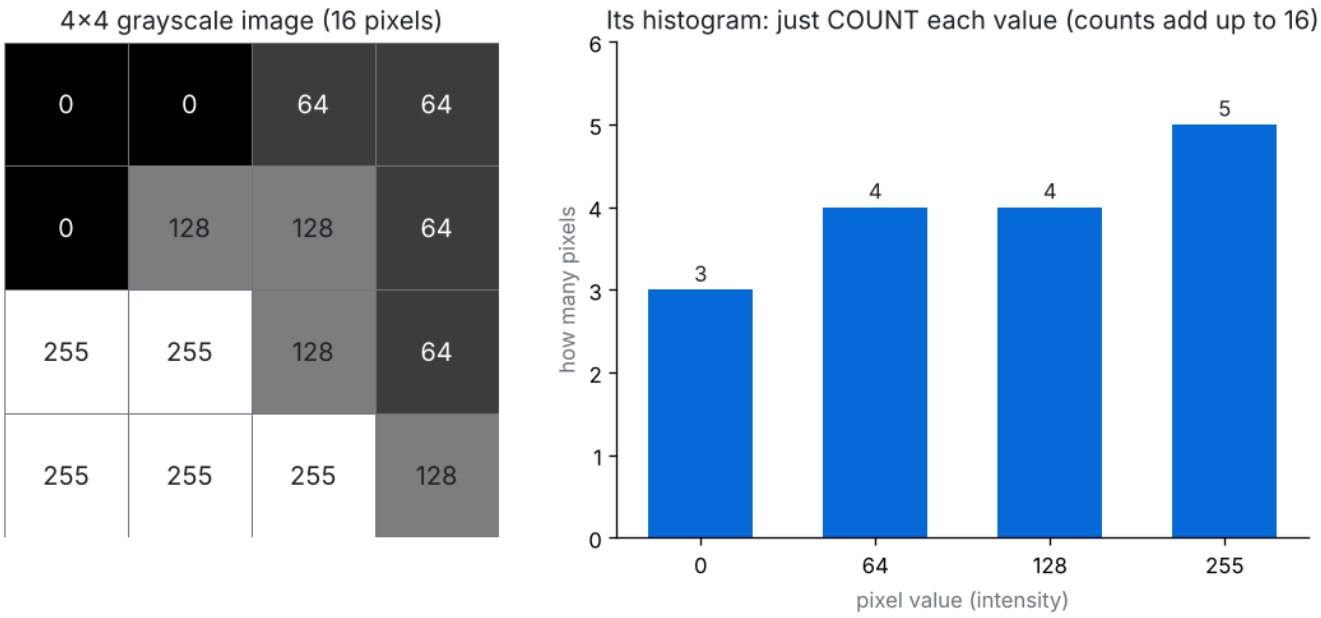
```
img = cv2.imread('sample.jpg', 0)     # 0 = cv2.IMREAD_GRAYSCALE
```

The `0` means "give me one brightness number per pixel". The result has shape `(rows, cols)`, with no channel number. Histograms and thresholding both work on grayscale.

---

## 1. Image histograms

> A **histogram** is a bar chart that **counts pixels**. The x-axis is the brightness value (0–255) and the y-axis is how many pixels have that value.

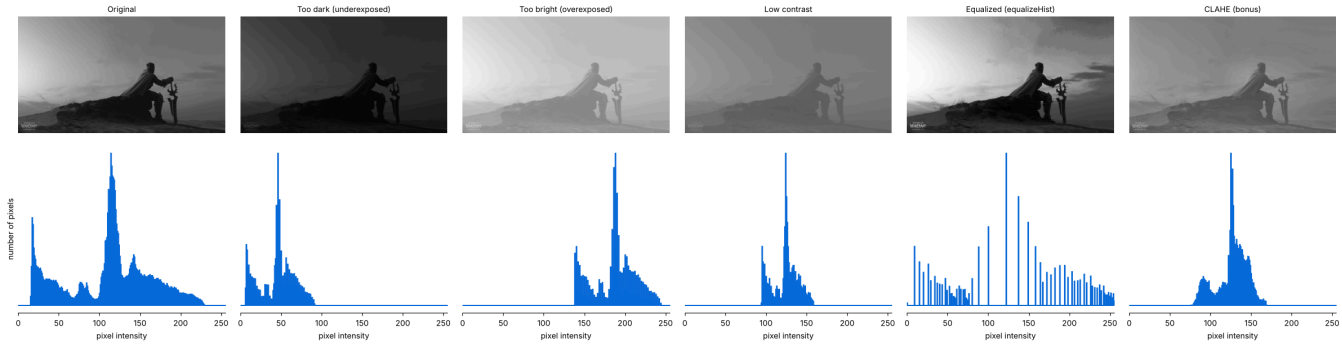



It throws away *where* the pixels are and keeps only *how many* of each brightness there are. Think of tipping a jar of coloured sweets onto a table and sorting them into piles by colour.

```
plt.hist(img.ravel(), 256, [0, 256])          # lecture: matplotlib counts for you
hist = cv2.calcHist([img], [0], None, [256], [0, 256])   # OpenCV: returns the counts
#                   images channel mask  bins   range
```

- `img.ravel()` flattens the 2-D grid into one long list of numbers.
- `256` = number of bars (**bins**); `[0, 256]` = the range covered (the end is excluded).
- `calcHist` arguments are all **lists**, so you can pass several images or channels. Channel `[0]` is the only channel of a grayscale image (for colour: 0 = B, 1 = G, 2 = R).
- The **mask** argument lets you count only part of the image ( `None` = the whole thing).

### Reading a histogram (exam favourite)




| Histogram looks like… | Image is… |
|---|---|
| Hump on the **left** | **Dark / underexposed** |
| Hump on the **right** | **Bright / overexposed** |
| **Narrow** hump in the middle | **Low contrast** (flat, greyish) |
| Spread across 0–255 | Good contrast |
| **Two separate humps** (bimodal) | Object + background → **perfect for thresholding** |

**Histogram equalization** ( `cv2.equalizeHist(img)` ) stretches a squashed histogram across the full 0–255 range to boost contrast. The Key Concepts sheet calls histograms "the basis for enhancement techniques like histogram equalization". **CLAHE** is a smarter version that equalizes small tiles separately.

▶ Run `code/day4_01_histograms.py` .

---

## 2. Thresholding: making an image binary

> **Thresholding** turns a grayscale image into a **binary** (black-and-white) image by asking every pixel one question: *"are you brighter than T?"* Yes → 255 (white). No → 0 (black).

```
ret, thresh = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
#  ↑ the T used    src  T    value-if-yes   type
```

- **Returns 2 things:** the threshold actually used ( `ret` ) and the image. The lecture writes `_, thresh_global = ...` because `ret` is just 127 here, which it already knows. With Otsu, `ret` is the useful answer.
- **Why binary?** It separates **foreground from background**, which is the first step for contours (Day 6), morphology (Day 5), OCR and measuring objects.

### The 5 threshold types




| Type | Pixel > T | Pixel ≤ T |
|---|---|---|
| `THRESH_BINARY` | 255 | 0 |
| `THRESH_BINARY_INV` | 0 | 255 (inverted, so **dark objects become white**) |
| `THRESH_TRUNC` | T (capped) | unchanged |
| `THRESH_TOZERO` | unchanged | 0 |
| `THRESH_TOZERO_INV` | 0 | unchanged |

---

## 3. Global thresholding

**One** fixed cut-off for the **whole** image: `cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)`.

- Simple and fast.
- **Fails when the lighting is uneven.** A shadow makes one side darker, and no single number works for both sides.

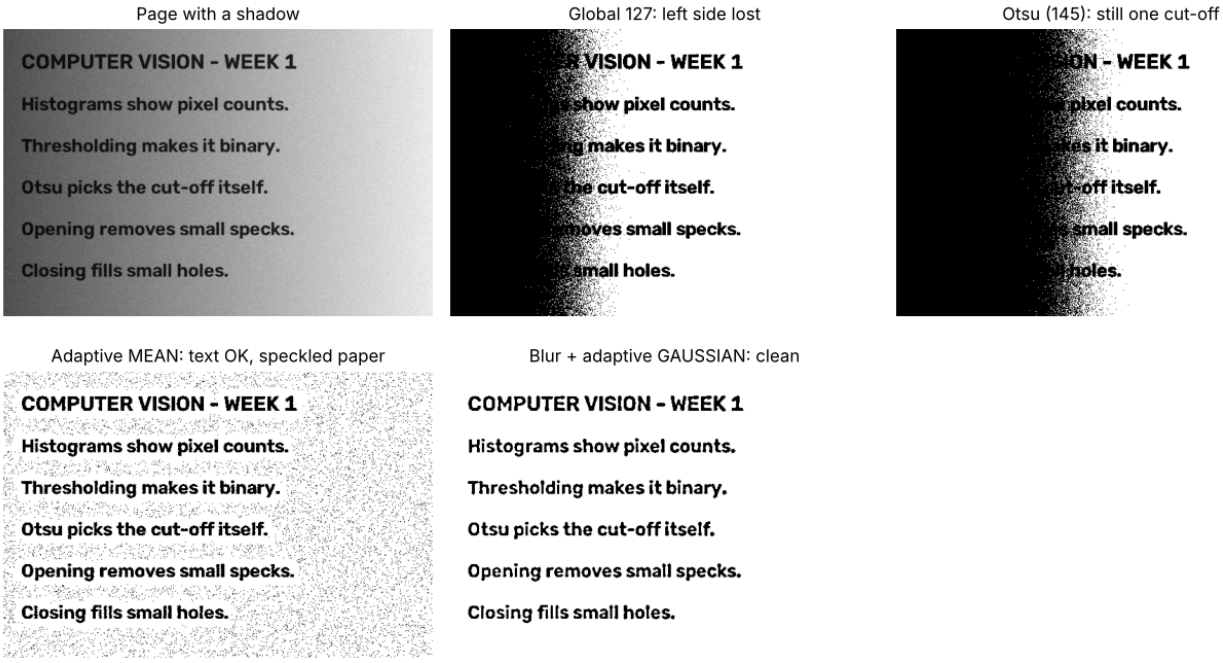
## 4. Adaptive thresholding

The threshold is **calculated separately for small regions**, so each neighbourhood gets its own cut-off. Use it for images with varying illumination.

```
thresh = cv2.adaptiveThreshold(img, 255,
                               cv2.ADAPTIVE_THRESH_MEAN_C,   # how to compute T
locally
                               cv2.THRESH_BINARY,
                               11,     # blockSize: neighbourhood size (ODD, > 1)
                               2)      # C: constant subtracted from the local mean
```

| Method | Local threshold T = |
|---|---|
| `ADAPTIVE_THRESH_MEAN_C` | plain **average** of the blockSize × blockSize neighbourhood − C |
| `ADAPTIVE_THRESH_GAUSSIAN_C` | **weighted** average (nearer pixels count more) − C |

Notice that `adaptiveThreshold` returns **only the image**, unlike `threshold`, which returns `(ret, image)`.

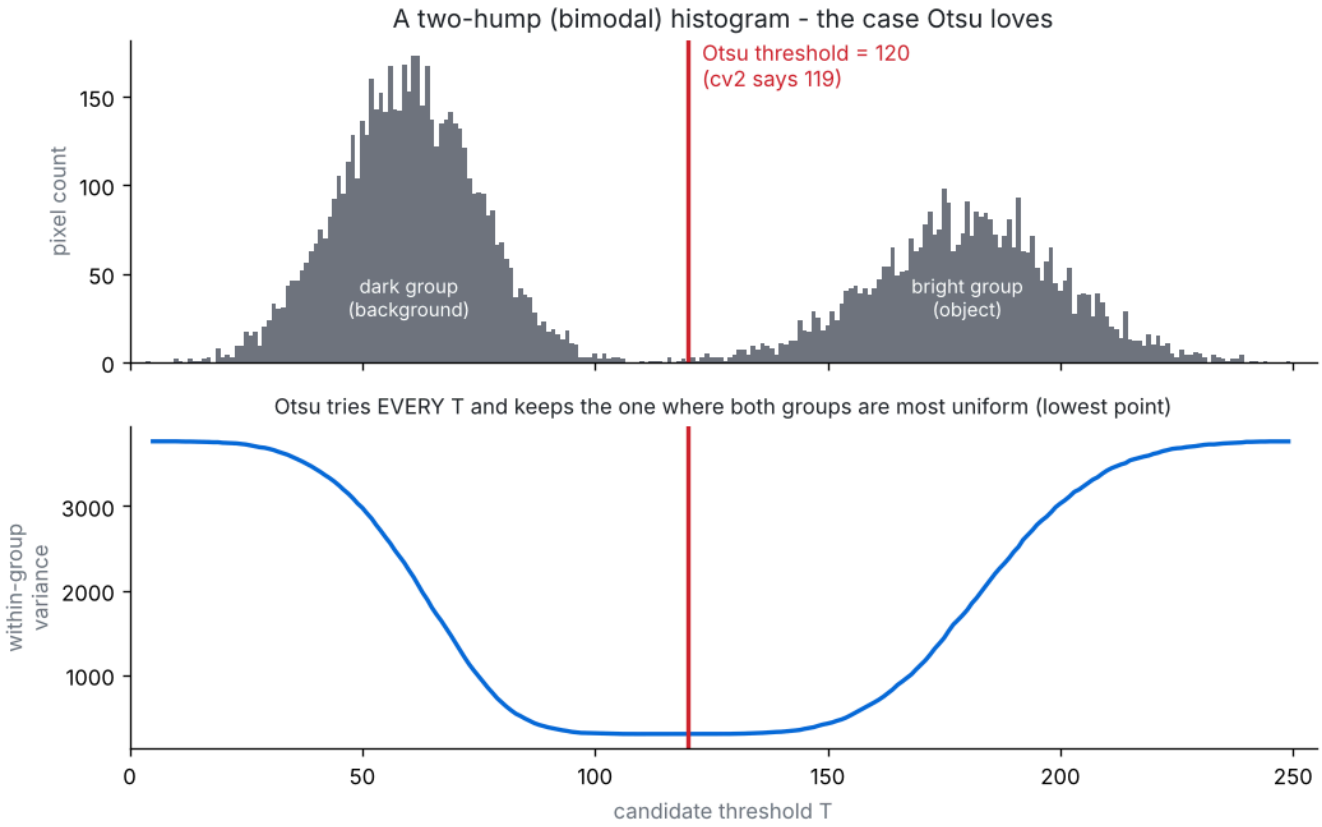



**Why the lecture's adaptive result looks like a pencil sketch:** `blockSize = 11` is tiny compared with a 5472×3648 photo. In big flat areas each 11×11 block is almost uniform, so tiny noise decides black or white. Blurring first, using a bigger `blockSize` or using a bigger `C` all fix it (see the last panel above).

## 5. Otsu's method

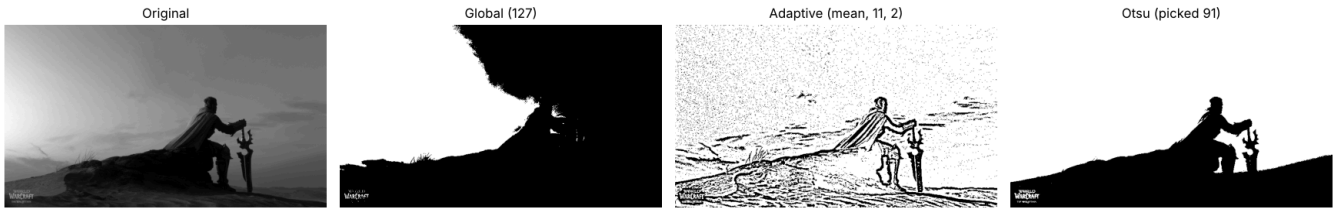
> **Otsu automatically calculates the optimal threshold value.** It tries every possible T and keeps the one that makes the two groups (below T and above T) as **uniform as possible inside themselves**, i.e. it **minimizes intra-class (within-group) variance**.

```
T, thresh_otsu = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
print(T)        # the threshold Otsu picked; the 0 you passed is ignored
```




- Works best with a **bimodal** (two-hump) histogram: object and background.
- It is still **one global** threshold, so it fails under uneven lighting just like a fixed 127.
- You can combine flags: `THRESH_BINARY_INV + THRESH_OTSU`.
- Blurring first (`GaussianBlur`) usually gives Otsu a cleaner result.

## Lab exercise: compare all three




On this photo Otsu picked T = 91. That separates the dark knight and rock from the bright sky better than the fixed 127.

| | Global | Adaptive | Otsu |
|---|---|---|---|
| Threshold | one, chosen by you | one **per neighbourhood** | one, chosen **automatically** |
| Uneven lighting | ❌ | ✅ | ❌ |
| Need to guess T? | yes | blockSize and C | no |
| Best for | controlled lighting | documents, shadows | bimodal histograms |

▶ Run `code/day4_02_thresholding.py`.

---

## 6. Applications

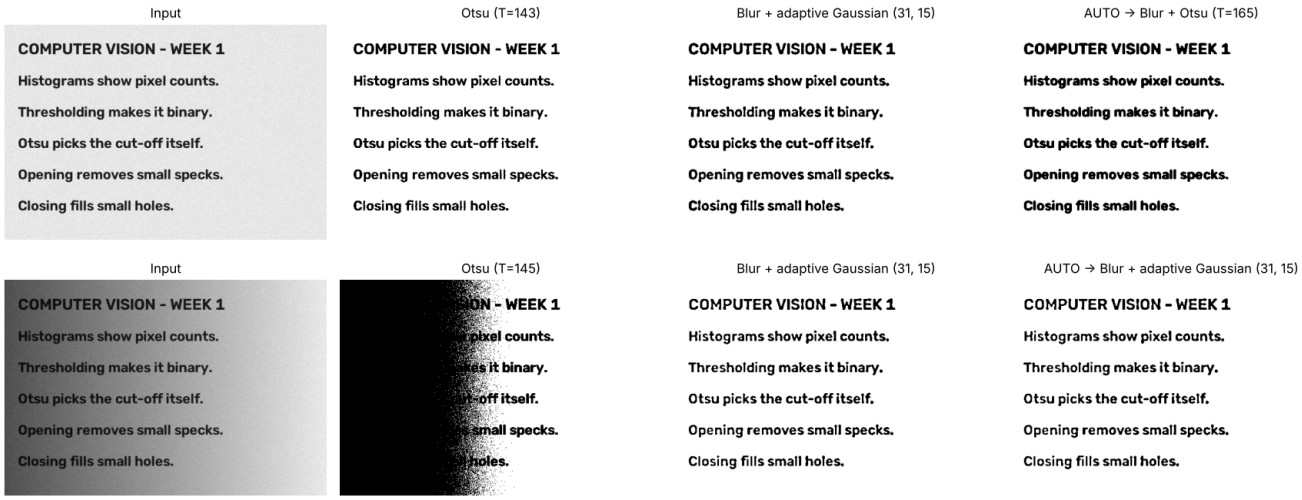
| Area | Use |
|---|---|
| Document scanning | **Binarization**: text vs paper |
| Medical imaging | **Tumour segmentation** (separate the region from the tissue) |
| Industrial inspection | **Defect detection** (scratches, spots) |
| Object detection | Separate foreground from background before contours |

## 7. Mini-project: Document Binarizer

**Task:** a scanned document goes in, and a clean binary version comes out using Otsu.

```
doc = cv2.imread('sample.jpg', 0)
_, doc_bin = cv2.threshold(doc, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

The lecture ran it on the Pantone *photo*, so the result was blotchy. On a real page, Otsu is excellent when the lighting is even. With a shadow you need adaptive thresholding. `day4_03` includes an **auto** mode that checks whether the lighting is uneven and picks the right method.




▶ Run `code/day4_03_mini_project_document_binarizer.py`.

---

## ✅ Day 4: check yourself

1. What does the y-axis of a histogram show? *(the number of pixels with each intensity)*
2. Histogram bunched on the left means? *(a dark / underexposed image)*
3. What does `cv2.threshold` return? *(a tuple: (threshold used, binary image))*
4. When does global thresholding fail, and what fixes it? *(uneven lighting; adaptive thresholding)*
5. What does Otsu minimise? *(intra-class (within-group) variance)*
6. In Otsu, why pass 0 as the threshold? *(it's ignored; Otsu computes T itself)*
7. Why must `blockSize` be odd? *(the neighbourhood needs a centre pixel)*

# Day 5 · Morphological Operations

**Lecture covers:** Dilation · Erosion · Opening & Closing · Structuring elements · Applications · Lab exercise · Mini-project: Noise removal in scanned documents

**Code for today:** `code/day5_01` → `day5_03`

---

## The big idea

**Morphology** = operations that change the **shape** of the white regions in an image. They're mainly used on **binary** images (after thresholding, Day 4), but they also work on grayscale.

The Key Concepts sheet's analogy is the best way to remember them. Think of editing a drawing with a **marker and an eraser**:

| Operation | Like… | Effect |
|---|---|---|
| **Dilation** | going over every stroke with a **thick marker** | white shapes get **fatter** |
| **Erosion** | **shaving** the edges of every stroke | white shapes get **thinner** |
| **Opening** | **erasing tiny specks** | small white noise removed |
| **Closing** | **filling tiny gaps** | small black holes filled |

> ⚠️ Morphology treats **WHITE (255) as the object**. If your objects are dark on light paper, invert first with `cv2.THRESH_BINARY_INV` or `cv2.bitwise_not(img)`. Otherwise "dilate" thickens the paper, not the text.

---

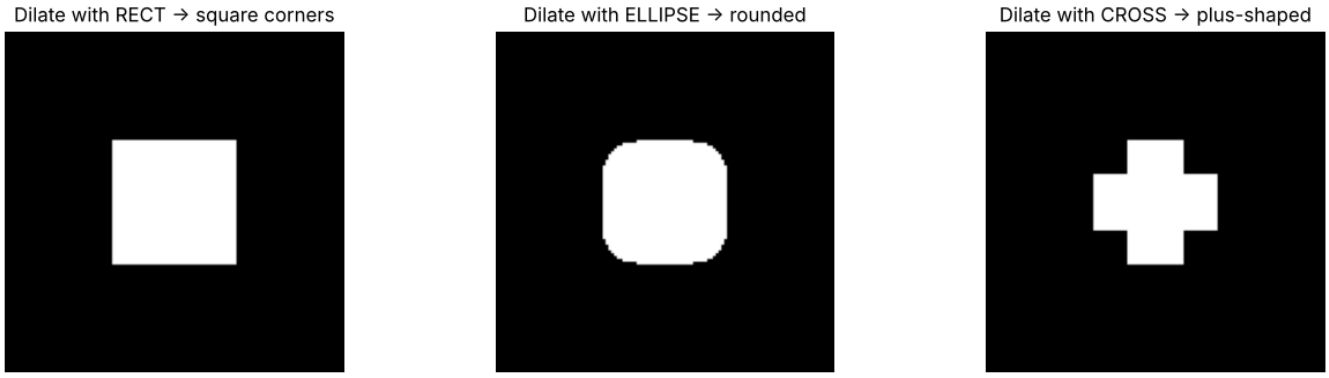
## 1. Structuring element (kernel)

A **small binary matrix used to probe the image**. It's the shape of the marker tip.

```
kernel = np.ones((5, 5), np.uint8)            # lecture: 5x5 square of 1s

cv2.getStructuringElement(cv2.MORPH_RECT,    (5, 5))   # square
cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))   # roughly round
cv2.getStructuringElement(cv2.MORPH_CROSS,   (5, 5))   # plus sign
```

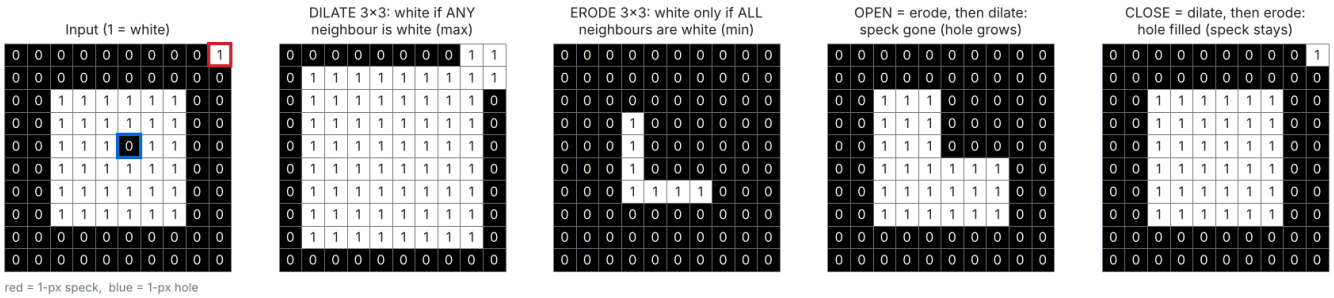
- **Shape**: rectangle, ellipse or cross. It decides the shape of the corners after dilation.
- **Size** decides how strong the effect is. A bigger kernel removes bigger specks but also damages more detail.
- **Rule of thumb:** the kernel should be *bigger than the noise* but *smaller than the thinnest part you want to keep*.




Unlike a convolution kernel (Day 3), there's no multiply-and-add here. The structuring element only says **which neighbours to look at**.

---

## 2. Dilation and erosion




| | Dilation | Erosion |
|---|---|---|
| Rule (binary) | pixel becomes white if **ANY** pixel under the kernel is white | pixel stays white only if **ALL** pixels under the kernel are white |
| Rule (grayscale) | **maximum** under the kernel | **minimum** under the kernel |
| White regions | **expand** | **shrink** |
| Use | **connect broken parts**, fill gaps in text characters | **remove small noise**, thin lines, separate touching objects |
| Code | `cv2.dilate(img, kernel, iterations=1)` | `cv2.erode(img, kernel, iterations=1)` |

`iterations=N` repeats the operation N times. That's like using a bigger kernel.

Input: text, thin line, cracked block, noise dots



Dilate 5×5: fatter, crack filled, dots grow



Erode 5×5: thinner, thin line + dots GONE



Dilate x3 iterations: much fatter



**Why the lecture's dilated and eroded photos barely change:** on a **grayscale photo**, dilation is just a local *max*, so the image gets slightly brighter and blockier. Erosion is a local *min*, so it gets slightly darker. Morphology shows its real power on **binary images with small details**.

▶ Run `code/day5_01_morphology_basics.py`.

---

## 3. Opening and closing

> **OPENING = Erosion, then Dilation.** It removes small white noise while preserving the shape of big objects. **CLOSING = Dilation, then Erosion.** It fills small holes inside objects.

```
opened = cv2.morphologyEx(img, cv2.MORPH_OPEN,  kernel)
closed = cv2.morphologyEx(img, cv2.MORPH_CLOSE, kernel)
```

**Why the second step?** Erosion alone removes specks but also shrinks everything. Dilating afterwards grows the survivors back to about their original size, and the specks don't come back because they were completely erased. Closing is the same idea in reverse.

**Memory trick:** *Opening* = **E**rode first, like "**O**ut with the specks". *Closing* = **D**ilate first, to "**C**lose the holes".



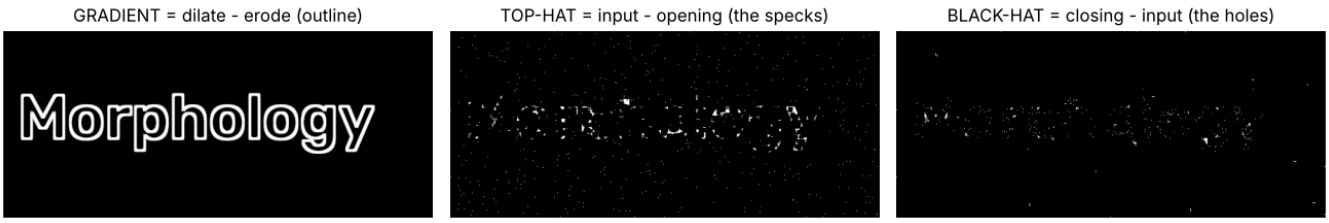




**Order matters** when an image has both problems. Above, opening first would widen the pinholes inside the letters. So: **close first** (seal the holes), **then open** (delete the specks).

### Bonus operations (built on the same pieces)

| Operation | = | Shows |
|---|---|---|
| `MORPH_GRADIENT` | dilation − erosion | the **outline** of shapes |
| `MORPH_TOPHAT` | input − opening | the small bright bits opening removed |
| `MORPH_BLACKHAT` | closing − input | the small dark holes closing filled |




▶ Run `code/day5_02_opening_closing.py`.

---

## 4. Applications

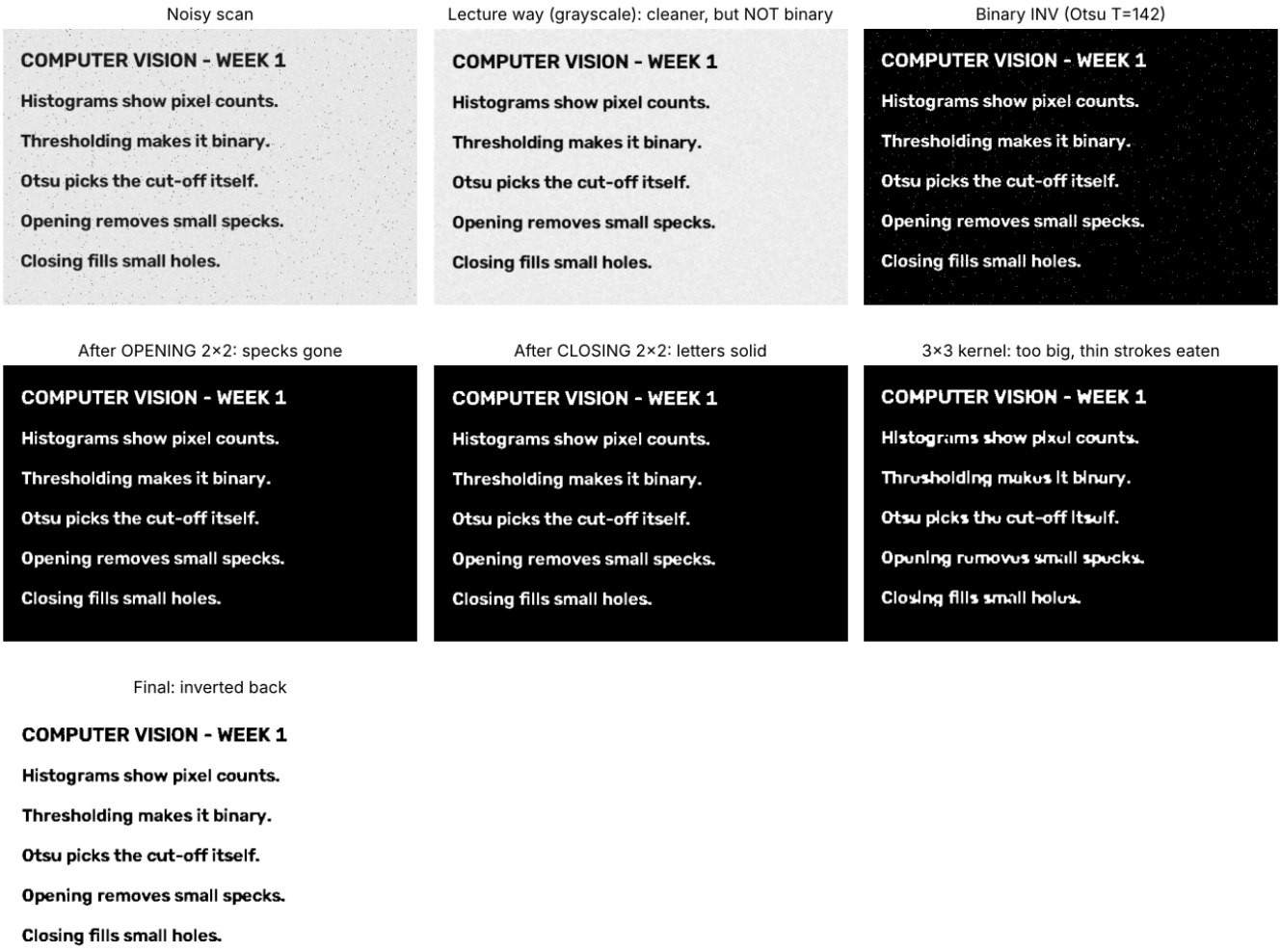
| Area | Use |
|---|---|
| Document processing | Remove noise, fill gaps in letters, make text more readable |
| Medical imaging | Highlight structures like cells or tissues |
| Industrial inspection | Detect defects in materials |
| Object detection | **Clean up shapes before contour analysis** (Day 6) |

## 5. Mini-project: Noise removal in scanned documents

**Task:** a noisy scanned document goes in, and a **cleaned binary** version comes out using opening and closing.

The lecture applied opening and closing to the grayscale photo, so "Cleaned Document" looks almost unchanged. The full pipeline is:

```
doc = cv2.imread('page.jpg', 0)
_, binary = cv2.threshold(doc, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)  #
text → WHITE
kernel = np.ones((2, 2), np.uint8)       # bigger than 1-px specks, smaller than 3-px
strokes
clean = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)    # remove specks
clean = cv2.morphologyEx(clean,  cv2.MORPH_CLOSE, kernel)   # fill gaps in letters
final = cv2.bitwise_not(clean)           # back to black text on white
```




Notice the "3×3 kernel" panel: it's **too big** for this thin text and eats the strokes ("pixel" → "pixul").

**A subtle point:** on a *grayscale* page with dark specks, it's **closing** (not opening) that removes the specks, because closing removes small *dark* things and opening removes small *bright* things. Inverting so the text is white removes the confusion.

▶ Run `code/day5_03_mini_project_document_cleaning.py`.

---

## ✅ Day 5: check yourself

1. Dilation vs erosion in one line each? *(dilation: white grows (local max); erosion: white shrinks (local min))*
2. Opening = ? Closing = ? *(erode then dilate; dilate then erode)*
3. Which removes salt noise (white specks)? Which fills small holes? *(opening; closing)*
4. What is a structuring element? Name 3 shapes. *(a small binary matrix that probes the image; rect, ellipse, cross)*
5. What happens if the kernel is too big? *(it destroys real detail, e.g. thin strokes and small objects)*
6. Why invert before morphology on black-on-white text? *(morphology treats white as the object)*
7. What does `iterations=3` do? *(repeats the operation 3 times, a stronger effect)*

# Day 6 · Contours & Shape Analysis

**Lecture covers:** Contour detection · Bounding boxes · Convex hull · Shape classification · Applications · Lab exercise · Mini-project: Shape Detector

**Code for today:** `code/day6_01` → `day6_03`

---

## 1. Contours

> **Contours are curves joining continuous points along the boundary** of objects with the same intensity or colour. A contour is the outline of one blob, stored as a list of (x, y) points.

Picture a coin on paper: tracing round it with a pencil gives you its contour.

**Pipeline** (this is Days 4 + 5 + 6 working together):

```
grayscale → threshold (Day 4) → [clean up with morphology (Day 5)] → findContours → measure / draw
```

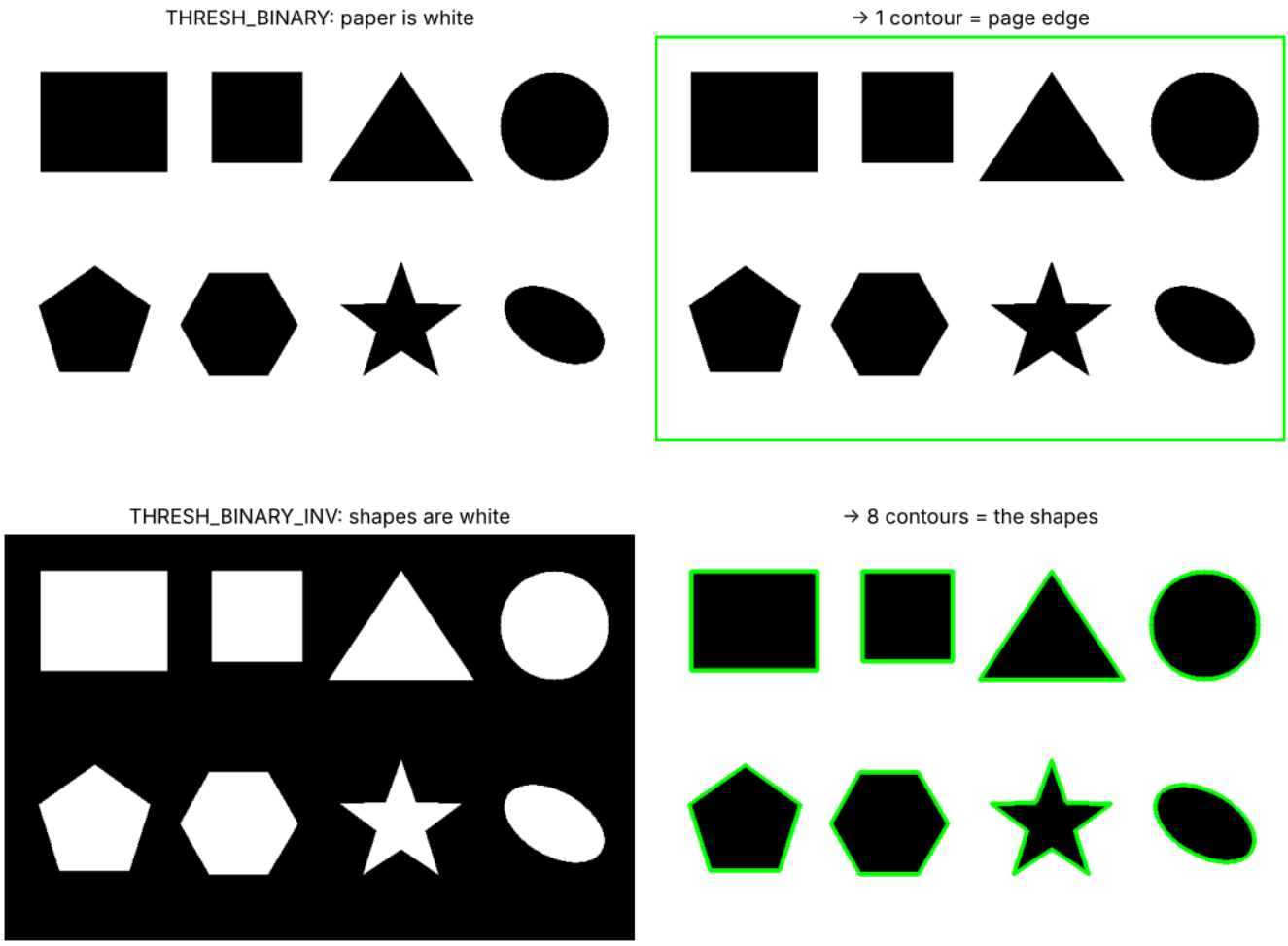
```
_, thresh = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY_INV)   # shapes → WHITE
contours, hierarchy = cv2.findContours(thresh, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

img_contours = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)   # colour copy so we can draw
in colour
cv2.drawContours(img_contours, contours, -1, (0, 255, 0), 2)
#                image          list     which  colour   thickness
```

| Argument | Meaning |
|---|---|
| `contours` | a Python **list**; each item is one outline (a NumPy array of points) |
| `hierarchy` | which contour is inside which (see below) |
| `drawContours(..., -1, ...)` | `-1` = draw **all**; `2` = draw only contour number 2 |
| thickness `-1` | **fill** the contour instead of outlining it |
| `COLOR_GRAY2BGR` | needed because you can't draw a *green* line on a 1-channel grey image |

### ⚠️ The #1 contour bug: white is the object

`findContours` finds **white** objects on a **black** background. The lecture used `THRESH_BINARY` on dark shapes on white paper, which makes the **paper** the object: you get one contour around the page edge. Use **`THRESH_BINARY_INV`** so the shapes are white.



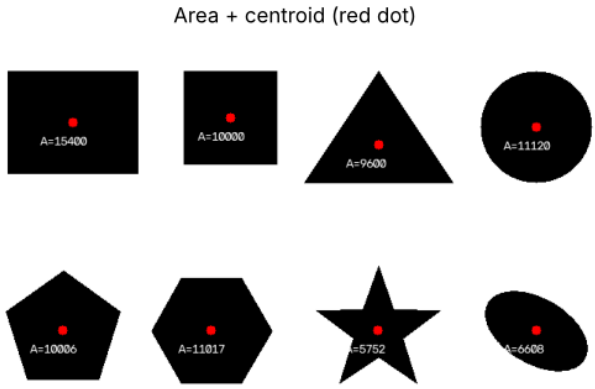

## Retrieval modes and approximation

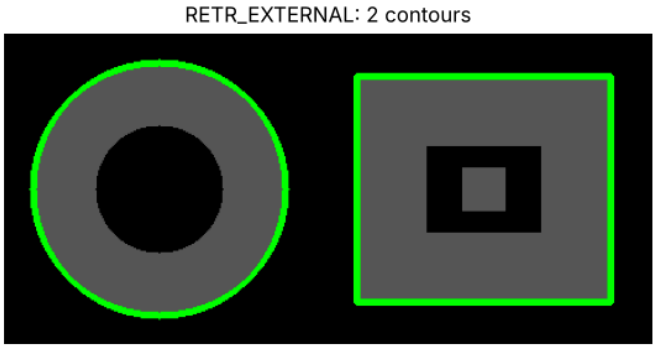
| Flag | Meaning |
|---|---|
| `RETR_EXTERNAL` | only the **outermost** outlines (ignores holes) |
| `RETR_LIST` | every outline, no parent/child info |
| `RETR_TREE` | every outline **plus** the full family tree (hierarchy) |
| `CHAIN_APPROX_NONE` | store **every** boundary point |
| `CHAIN_APPROX_SIMPLE` | straight runs are stored by their **end points** only (a rectangle → 4 points) |

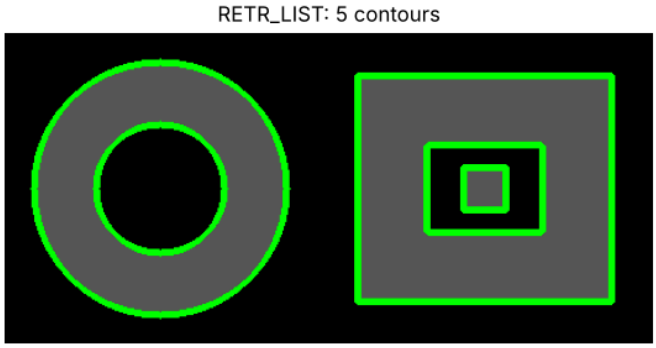
`hierarchy[0][i] = [next, previous, first_child, parent]`, where `-1` means "none".

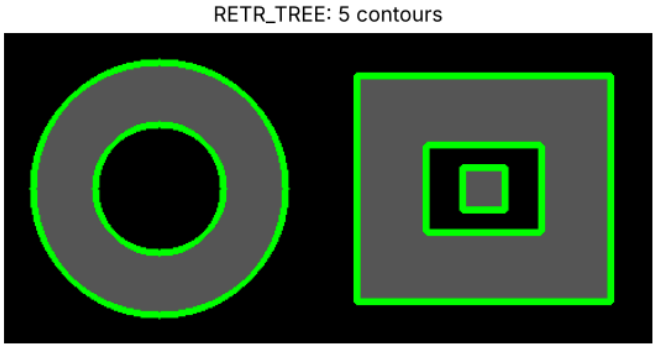
## Measuring contours: shape descriptors

| Measure | Code | Meaning |
|---|---|---|
| Area | `cv2.contourArea(cnt)` | pixels inside |
| Perimeter | `cv2.arcLength(cnt, True)` | outline length (`True` = closed) |
| Centroid | `M = cv2.moments(cnt)`; `cx = M['m10']/M['m00']`, `cy = M['m01']/M['m00']` | centre of mass |
| Aspect ratio | `w / h` from `boundingRect` | square ≈ 1 |
| Solidity | `area / hull area` | 1 = no dents; a star ≈ 0.5 |
| Circularity | `4π·area / perimeter²` | 1 = perfect circle |



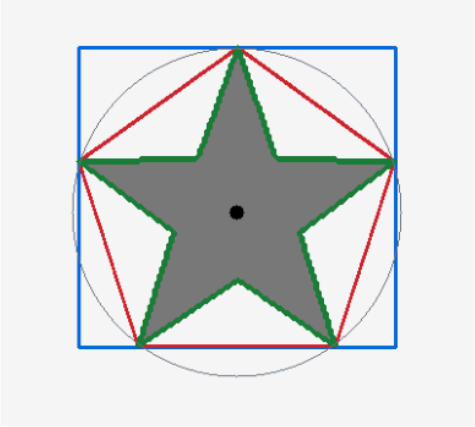




▶ Run `code/day6_01_contours.py`.

---

## 2. Bounding boxes, convex hull and more



```
contour (green): the outline points  -> findContours / drawContours
bounding box (blue): x, y, w, h        -> boundingRect
convex hull (red): rubber band around it -> convexHull
min enclosing circle (grey)            -> minEnclosingCircle
centroid (black dot): centre of mass    -> moments
area 5752 px, perimeter 527 px, solidity = area / hull area = 0.50
```

### Bounding box

> The **smallest (upright) rectangle that encloses a contour**. Used for **object localization**: "the object is here".

```
x, y, w, h = cv2.boundingRect(cnt)                      # (x, y) = top-left corner
cv2.rectangle(img, (x, y), (x + w, y + h), (255, 0, 0), 2)   # (255,0,0) in BGR =
BLUE
```

Variants: `cv2.minAreaRect(cnt)` = the tightest **rotated** rectangle (draw it with `cv2.boxPoints`). `cv2.minEnclosingCircle(cnt)` = the smallest circle around the shape.

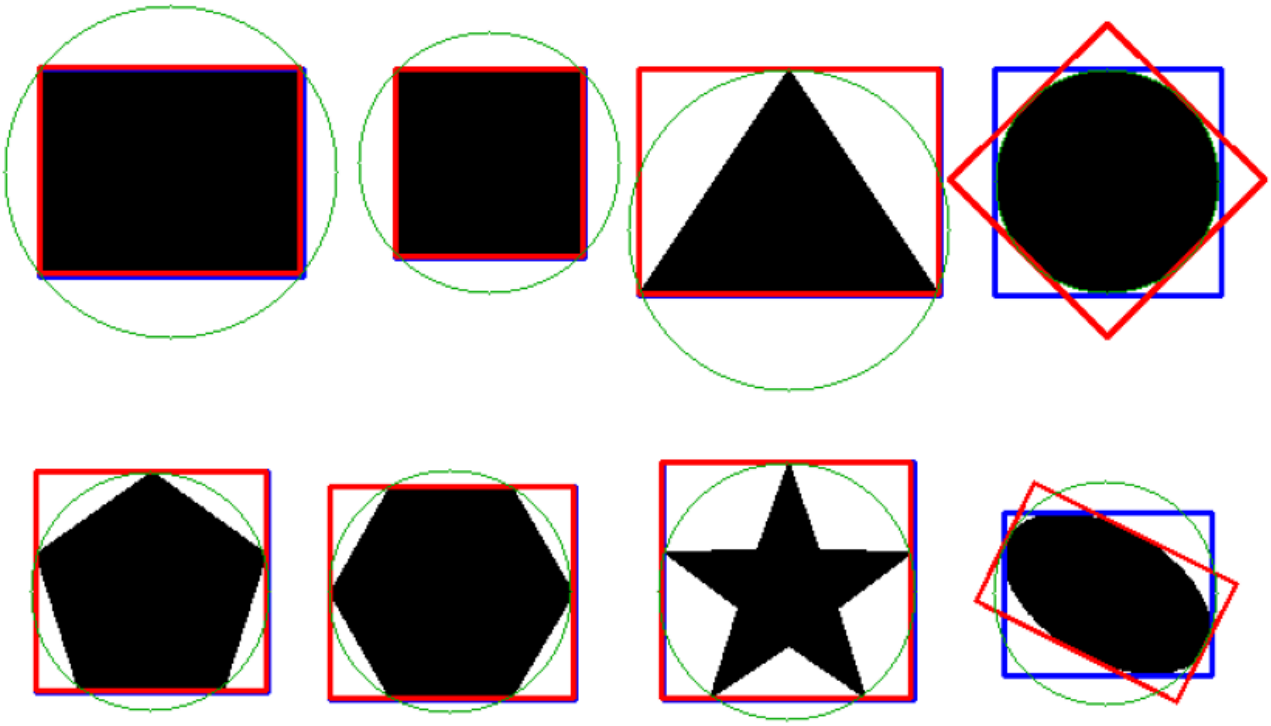
### Convex hull

> The **smallest convex polygon enclosing a contour**. Picture a rubber band snapped around the shape. Used for **shape simplification**.

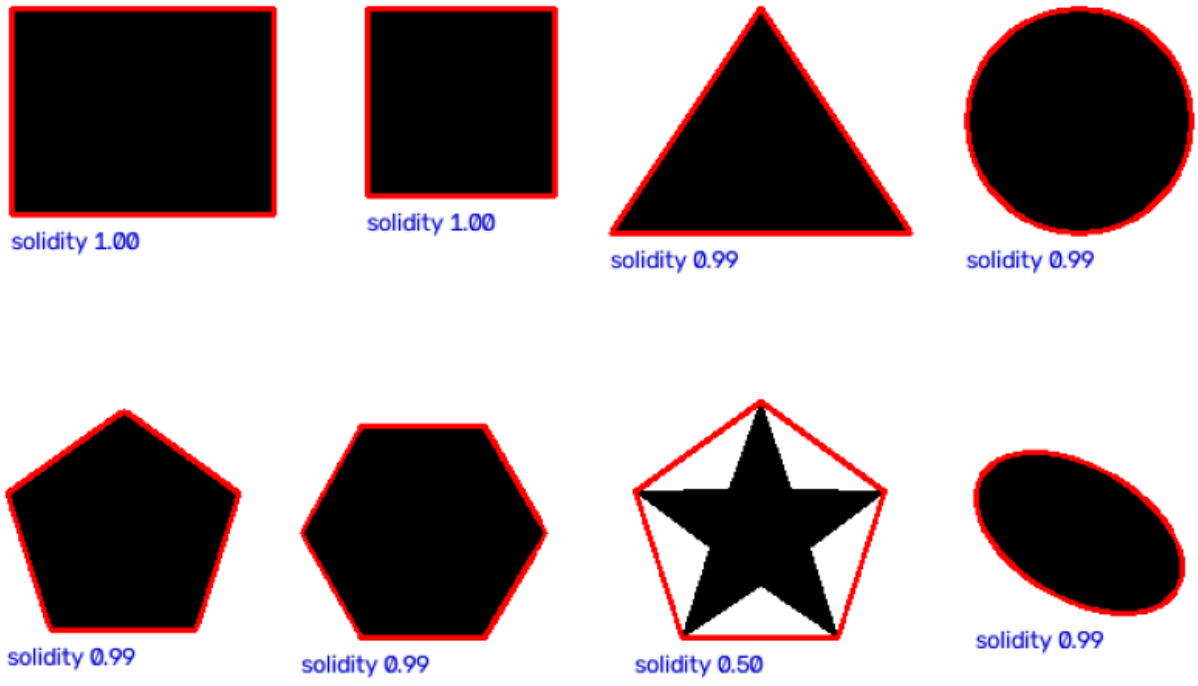
```
hull = cv2.convexHull(cnt)
cv2.drawContours(img, [hull], 0, (0, 0, 255), 2)         # note the [ ]: drawContours
wants a LIST
```

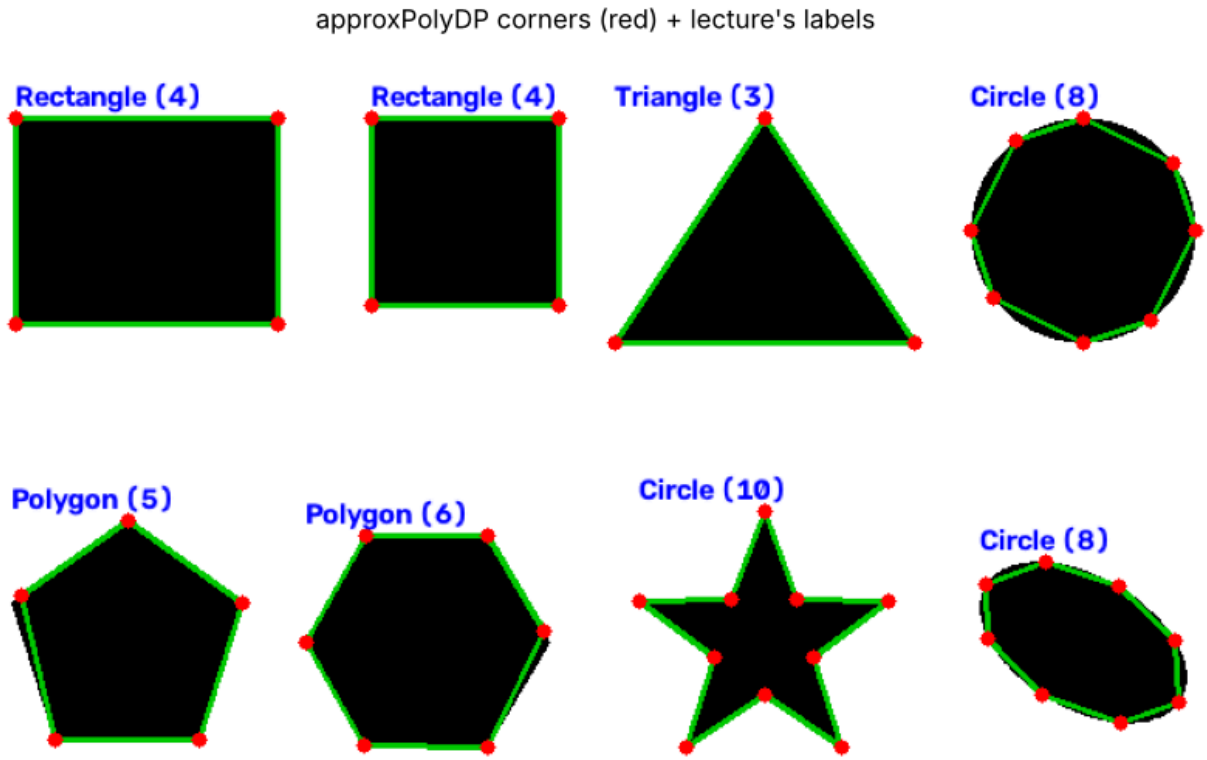
- **Convex** = no dents: every corner points outward. A square, circle or hexagon is convex. A **star is not**.
- The gaps between the shape and its hull are called **convexity defects**. They're used to count fingers in hand-gesture recognition, for example.
- `cv2.isContourConvex(cnt)` returns True or False.

boundingRect (blue), minAreaRect (red), minEnclosingCircle (green)



Convex hulls - note the star





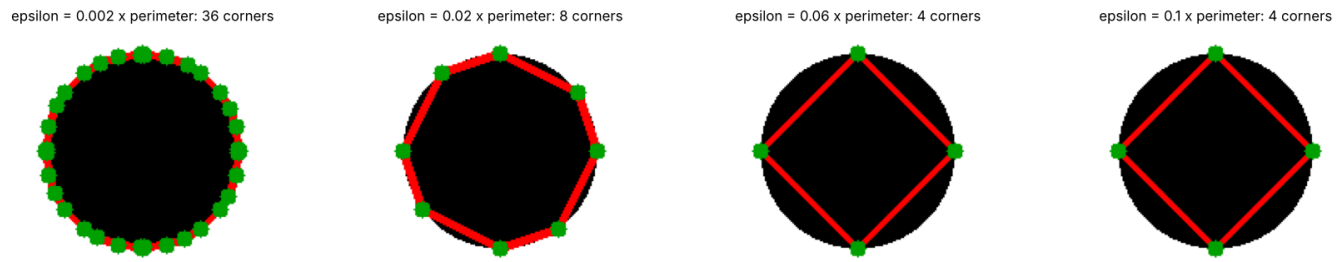

---

## 3. Shape classification

> **Approximate contours to polygons, then count the vertices.**

```
approx = cv2.approxPolyDP(cnt, 0.02 * cv2.arcLength(cnt, True), True)
n = len(approx)          # number of corners
# 3 → Triangle   4 → Rectangle   > 6 → Circle   otherwise → Polygon   (lecture's
rule)
```

`approxPolyDP` (Douglas–Peucker algorithm) simplifies a wobbly outline into a polygon with fewer corners. **epsilon** is how far the simplified line may stray from the real outline. The lecture uses **2% of the perimeter**.



A bigger epsilon gives fewer corners: at 6%, even a circle becomes a 4-cornered "rectangle". The lecture's simple rule also mislabels shapes. The star has 10 corners, so it's called "Circle", and the pentagon and hexagon are just "Polygon". Better rules:

| Shape | Test |
|---|---|
| Square vs rectangle | 4 corners and aspect ratio `w/h` ≈ 1 |
| Pentagon / hexagon | 5 / 6 corners |
| Star | **solidity < 0.8** (big dents) |
| Circle vs ellipse | many corners and circularity > 0.85 (circle) or lower (ellipse) |

▶ Run `code/day6_02_boxes_hull_shapes.py`.

---

## 4. Applications

| Area | Use |
|---|---|
| Object detection and tracking | Find, box and follow objects |
| Industrial inspection | Check part shape and size, find missing pieces |
| Medical imaging | **Cell shape analysis** (area, roundness) |

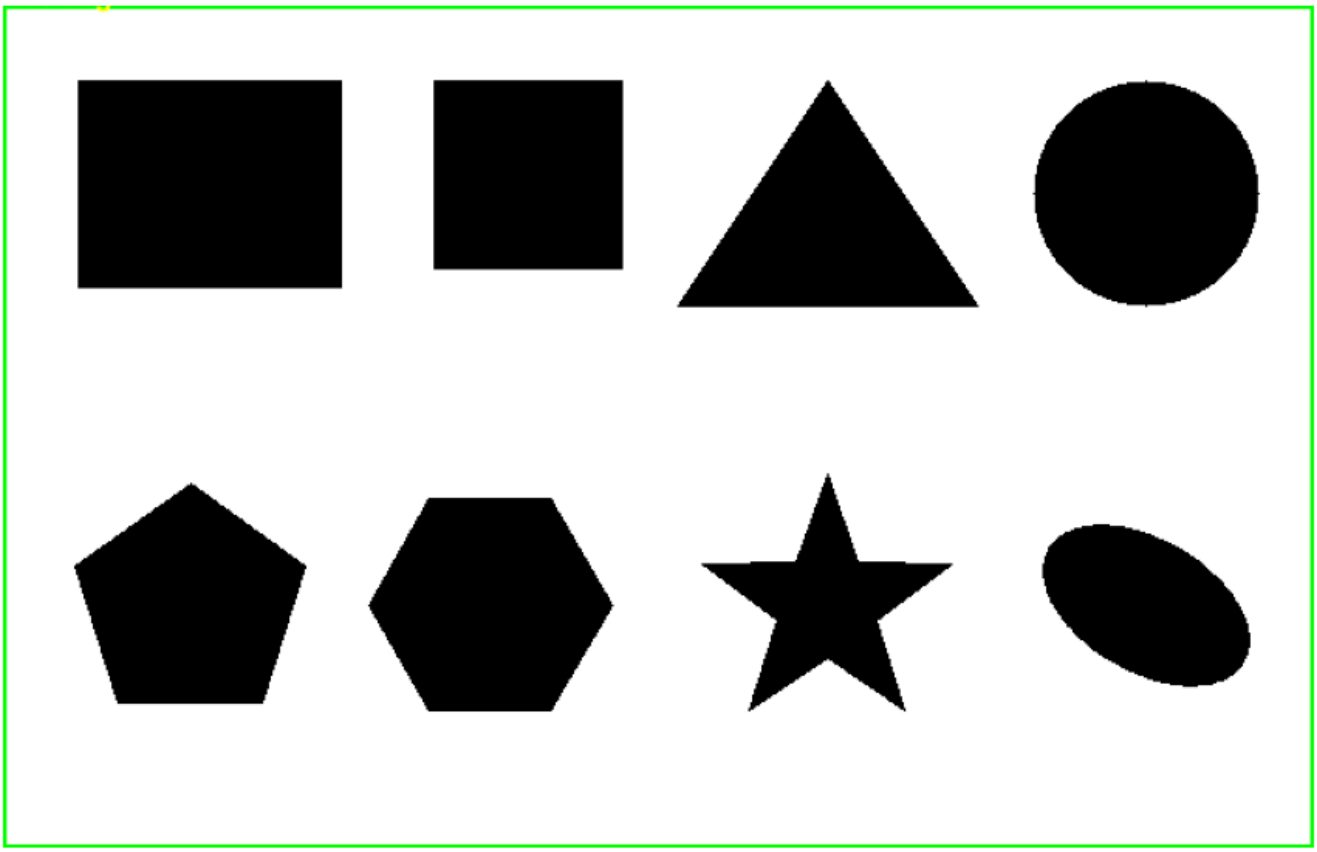
## 5. Lab exercise and mini-project: Shape Detector

**Task:** an image with multiple geometric shapes goes in, and contours, bounding boxes, convex hulls and labels come out.

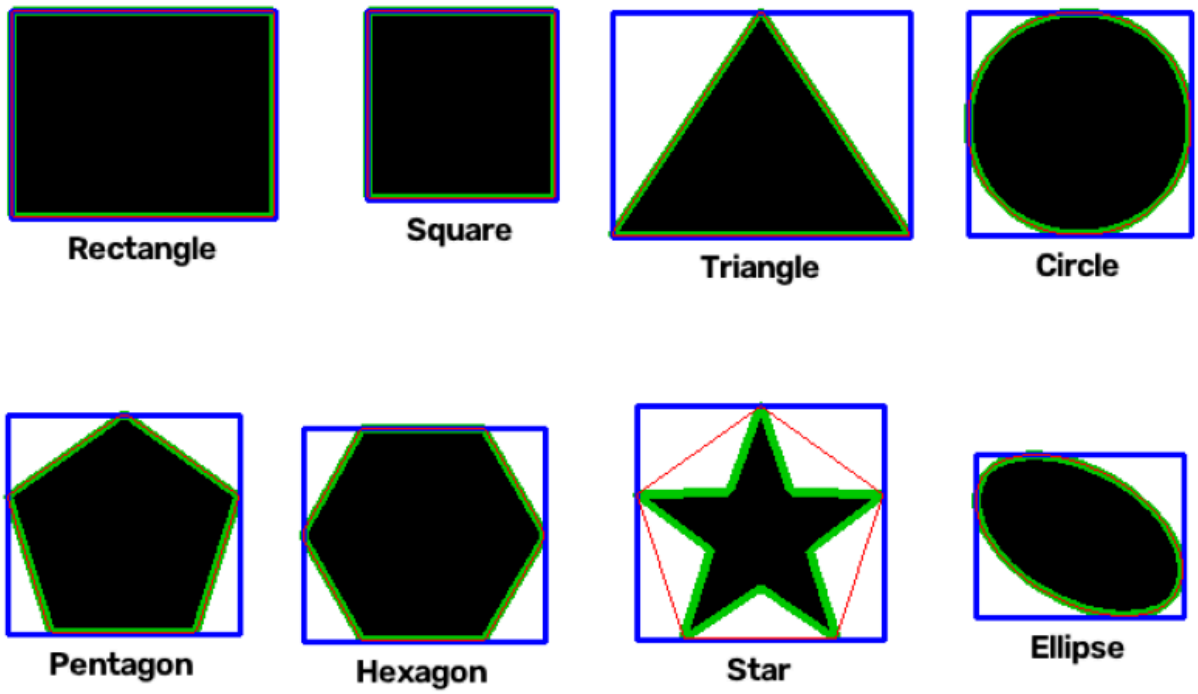
The lecture's `sample.png` is actually the same Pantone **photo**, not a shapes image. So its "Detected Shapes" output is covered in hundreds of tiny contours on noise and text. On a real shapes image the lecture's code finds **1 contour** (the page) because of the `THRESH_BINARY` bug. The fixed version:

1. Uses **BINARY_INV + Otsu** so the shapes are white.
2. **Ignores tiny contours** (`if cv2.contourArea(cnt) < 300: continue`).
3. Draws the contour, box, hull and label.
4. Uses the better rules from the table above.

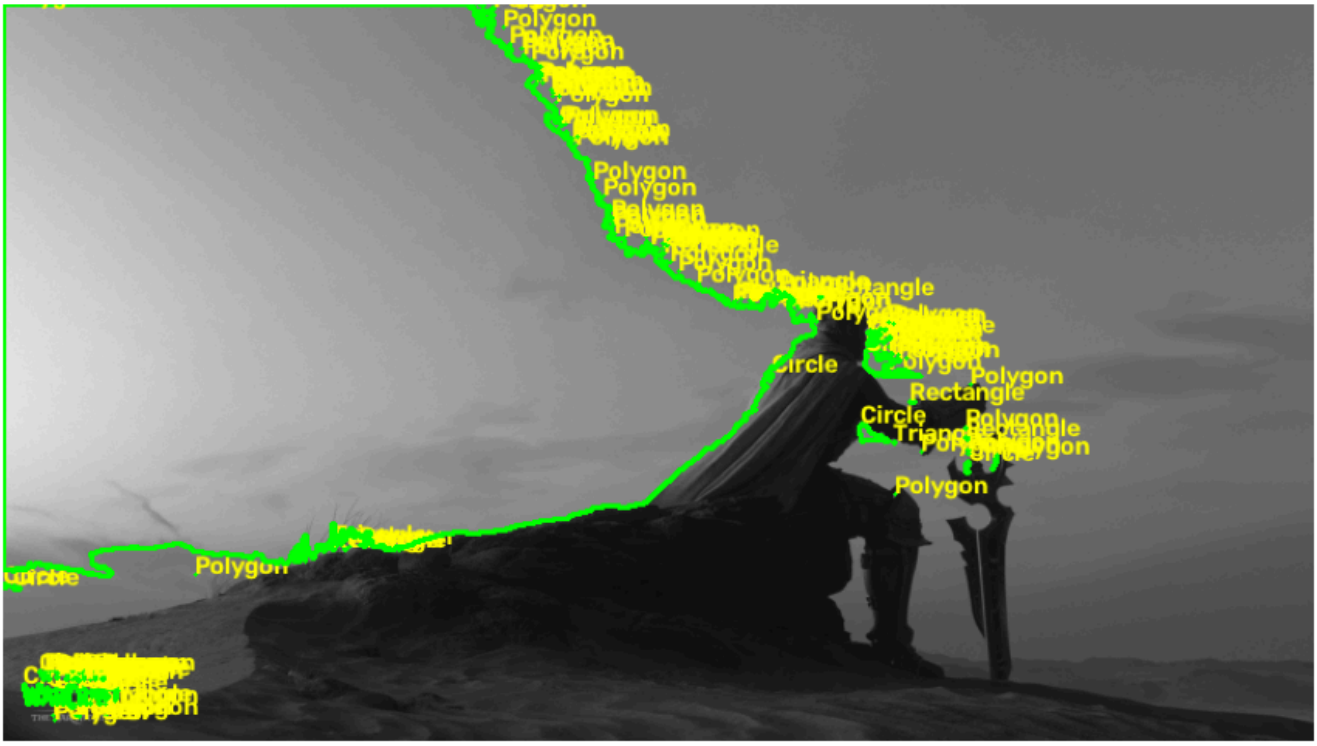
Lecture version on shapes: 1 contour



Improved: 8 shapes labelled



Lecture version on a photo: 123 contours




▶ Run `code/day6_03_mini_project_shape_detector.py`.

---

## ✅ Day 6: check yourself

1. What is a contour? *(a curve joining continuous boundary points of a shape)*
2. Why `THRESH_BINARY_INV` for dark shapes on white paper? *(findContours treats white as the object)*
3. What does `boundingRect` return? *(x, y, w, h: top-left corner, width, height)*
4. What is a convex hull? Is a star convex? *(the smallest convex polygon around the shape; no)*
5. In `approxPolyDP(cnt, 0.02*peri, True)`, what does 0.02 control? *(epsilon, the allowed deviation; bigger means fewer corners)*
6. How do you compute the centroid? *(moments: cx = m10/m00, cy = m01/m00)*
7. `RETR_EXTERNAL` vs `RETR_TREE`? *(outer contours only vs all contours plus the hierarchy)*

# Week 1 · Key Concepts (the summary sheet, explained)

This note covers **CV-Week-1-KeyConcepts.pdf**, the course's own summary of the week, plus the extra notebooks in `Week1.zip` (`imagerepresentation`, `colorimgrepresent`, `Convolution`).

The sheet has two halves: - **Part A** recaps Days 1–6. If you've done the day notes, this is revision. - **Part B** introduces **advanced topics** that have no lab: corners, SIFT, SURF, HOG, stereo vision, epipolar geometry, optical flow, Kalman filters and object tracking. Expect **"what is X / what is it used for"** questions. Each one has a runnable demo in `code/keyconcepts_01` and `keyconcepts_02`.

---

# Part A · Recap of Days 1–6

### Images as matrices (Day 1)

- **Grayscale image → 2-D matrix.** Each pixel is one number, 0 = black and 255 = white. Shape `(rows, cols)`.
- **Colour image → 3-D matrix** with 3 channels. Shape `(rows, cols, 3)`. Pixel `[255, 0, 0]` is pure red in RGB, but **pure blue in OpenCV's BGR**.
- Origin `(0, 0)` = **top-left**. x goes right, y goes **down**. Access a pixel with `image[y, x]`.
- `cv2.imread('sample.jpg', 0)` → grayscale (2-D). `cv2.imread('sample.jpg')` → colour (3-D, BGR).

The course notebook `imagerepresentation.ipynb` draws a 5×5 matrix as an image with the numbers on top. `colorimgrepresent.ipynb` does the same for a 3×3 RGB image: red, green, blue, yellow, cyan, magenta, grey, white, black. `code/day1_02_pixels_by_hand.py` covers the same idea.

### HSV (Day 1)

- **Hue** = the *type* of colour, an **angle on a colour wheel**: 0° = red, 120° = green, 240° = blue. *(OpenCV halves it: 0–179, so green = 60 and blue = 120.)*
- **Saturation** = how intense or pure: 0% = grey, 100% = vivid. *(OpenCV: 0–255.)*
- **Value** = brightness: 0% = black, 100% = brightest. *(OpenCV: 0–255.)*
- **Why it's useful:** it's closer to how humans think about colour, and you can pick out a hue range ("yellow", "red") **without worrying about brightness changes**. The sheet's example is detecting ripe fruit.

### Kernels and convolution (Day 3)

- **Kernel** (also called a *filter*, *mask* or *convolution matrix*) = a small grid of numbers (3×3, 5×5, 7×7) that says how a pixel and its neighbours are combined. **Larger kernels capture more context but are slower.**

- **Operation:** place the kernel over a pixel, multiply each kernel value by the pixel under it, and **sum** to get the new pixel value.
- **Formula:** $(I * K)(x, y) = \Sigma_m \Sigma_n I(x - m, y - n) \cdot K(m, n)$, where I = image and K = kernel. *(The PDF prints it as "I(x, −my − n)", which is a typesetting slip for I(x − m, y − n).)*

| Kernel | Matrix | Effect |
|---|---|---|
| Identity | `[[0,0,0],[0,1,0],[0,0,0]]` | unchanged |
| Box blur | `1/9 · ones(3,3)` | smooths, reduces detail |
| Sharpen | `[[0,-1,0],[-1,5,-1],[0,-1,0]]` | enhances edges and fine detail |
| Sobel X | `[[-1,0,1],[-2,0,2],[-1,0,1]]` | **vertical** edges |
| Sobel Y | `[[-1,-2,-1],[0,0,0],[1,2,1]]` | **horizontal** edges |
| Edge (from `Convolution.ipynb`) | `[[-1,-1,-1],[-1,8,-1],[-1,-1,-1]]` | edges in all directions (a Laplacian-style kernel; sum = 0) |

- **Why kernels matter:** feature extraction (edges, corners, textures), image enhancement (sharpen, smooth, emboss), and **deep learning**: in CNNs the kernels are *learned automatically*.
- `Convolution.ipynb` also shows a 3×3 example: blurring `[[10,20,30],[40,50,60],[70,80,90]]` with the 1/9 box kernel turns the centre 50 into **50** (the average of all 9 values). Check: (10+20+…+90)/9 = 450/9 = 50.

## Histograms and thresholding (Day 4)

- **Histogram:** how many pixels have each intensity. x-axis = intensity 0–255, y-axis = count. Uses: check brightness and contrast, spot **under- or over-exposure**, and it's the basis for **histogram equalization**.
- **Thresholding:** intensity **> T → 255 (white)**, **≤ T → 0 (black)**. Global = one T for the whole image. Adaptive = a different T per region (for varying lighting).
- **Applications:** document scanning (text vs background), medical imaging (highlight tumours), object detection (foreground vs background).

## Morphology (Day 5)

Dilation (expand white, connect broken parts, fill gaps in text), erosion (shrink white, remove small noise, thin lines), opening (erode → dilate, cleaning scans), closing (dilate → erode, filling holes in letters). Applications: documents, medical images, industrial inspection, **cleaning shapes before contour analysis**.

## Contours (Day 6)

Curves joining continuous boundary points of objects with the same intensity. Key descriptors: **area, perimeter, centroid, bounding box, convex hull**. Used for shape recognition, object detection and classification.

### Edge detection (Days 1 & 3)

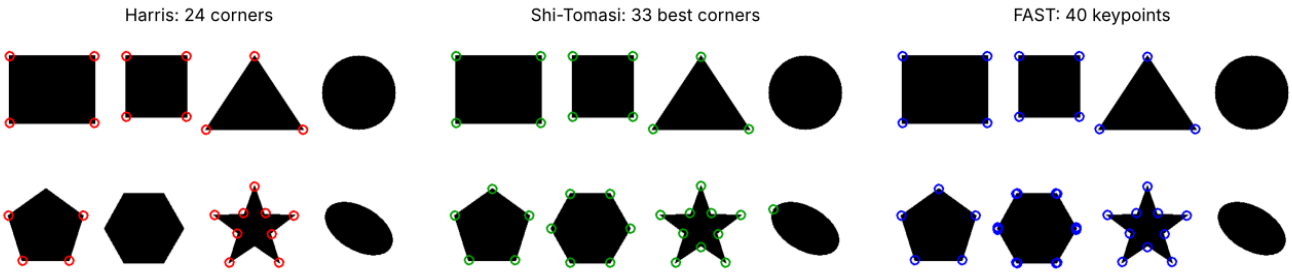
- **Purpose:** find boundaries where intensity changes sharply. **Methods:** Sobel, Canny, Laplacian. **Uses:** object segmentation, lane detection, medical imaging.
- **Edges are distinctive but not unique.** Slide a small window *along* an edge and every position looks the same, so an edge point is hard to pin down exactly. That's the motivation for corners ↓

---

# Part B · Advanced topics

## Corner detection

> **Corners are points where edges meet:** regions where intensity varies in **multiple directions**.




| Region | Slide a small window around… | Good landmark? |
|---|---|---|
| Flat | nothing changes | ❌ |
| Edge | changes only *across* the edge | ⚠️ ambiguous along it |
| **Corner** | changes in **every** direction | ✅ **repeatable and uniquely localizable** |

| Method | Idea | OpenCV |
|---|---|---|
| **Harris** | scores every pixel by how much a window changes when shifted in all directions | `cv2.cornerHarris(np.float32(gray), 2, 3, 0.04)` |
| **Shi-Tomasi** | improved Harris scoring; returns the N best corners ("good features to track") | `cv2.goodFeaturesToTrack(gray, N, quality, minDist)` |
| **FAST** | very quick test: is there a long arc of brighter or darker pixels on a 16-pixel ring? | `cv2.FastFeatureDetector_create()` |

**Applications:** image matching, tracking, **panorama stitching**.

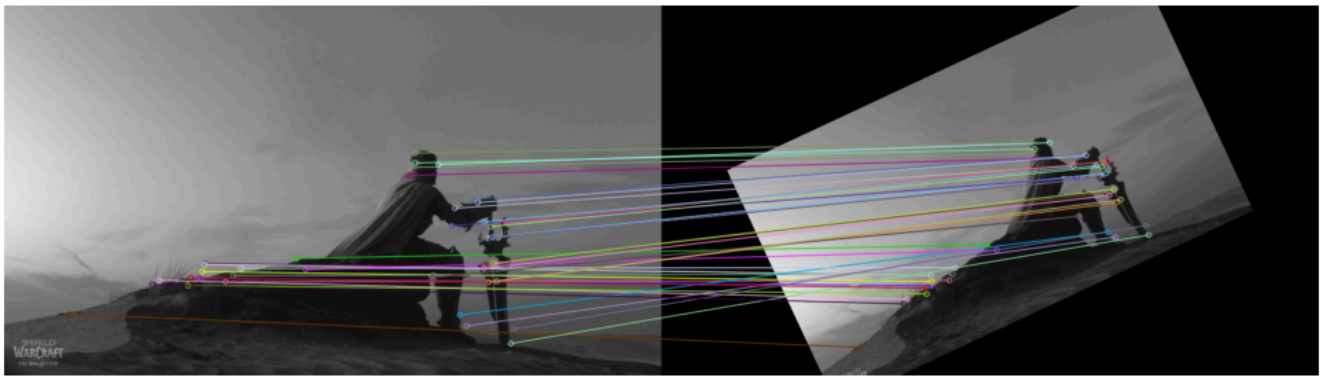
## SIFT: Scale-Invariant Feature Transform

> **Detect and describe** keypoints so they can be found again even if the image is **resized or rotated**.

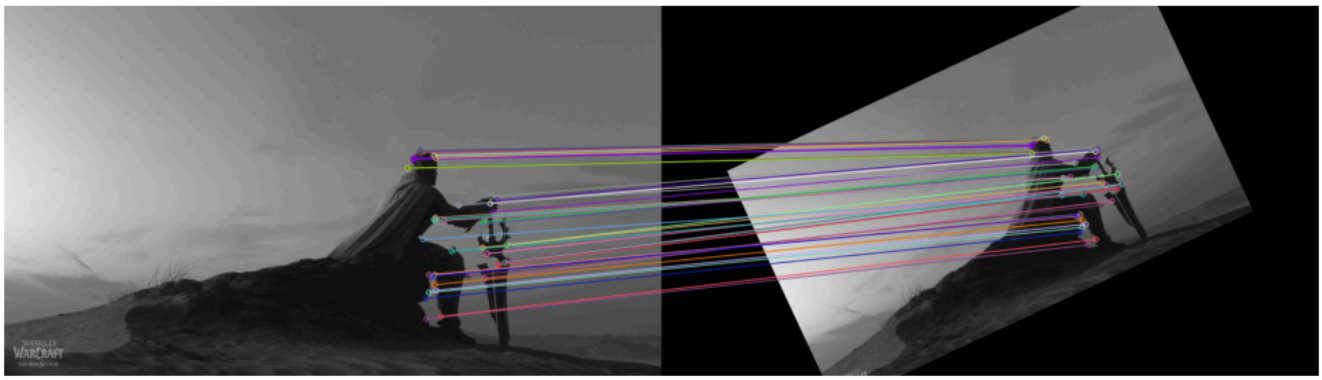
**Pipeline (learn the 4 steps):** 1. Build a **Gaussian scale space**: blur the image more and more, at several sizes. 2. Detect **extrema** in the **Difference of Gaussians** (DoG, one blur minus the next). These are blob-like keypoints at the right scale. 3. **Assign an orientation** (the dominant gradient direction), so rotation doesn't matter. 4. Build a **128-dimensional descriptor vector**: a "fingerprint" of the area around the keypoint.

- **Applications:** object recognition, image stitching, 3D reconstruction.
- **Strength:** robust to scale and rotation. **Limit:** computationally heavy.

SIFT: 40 best matches between original and rotated+shrunk copy



ORB: same idea, faster, free



Matching = for each fingerprint, find the closest fingerprint in the other image. **Lowe's ratio test** keeps a match only if the best match is clearly better than the second best (distance < 0.75 × second).

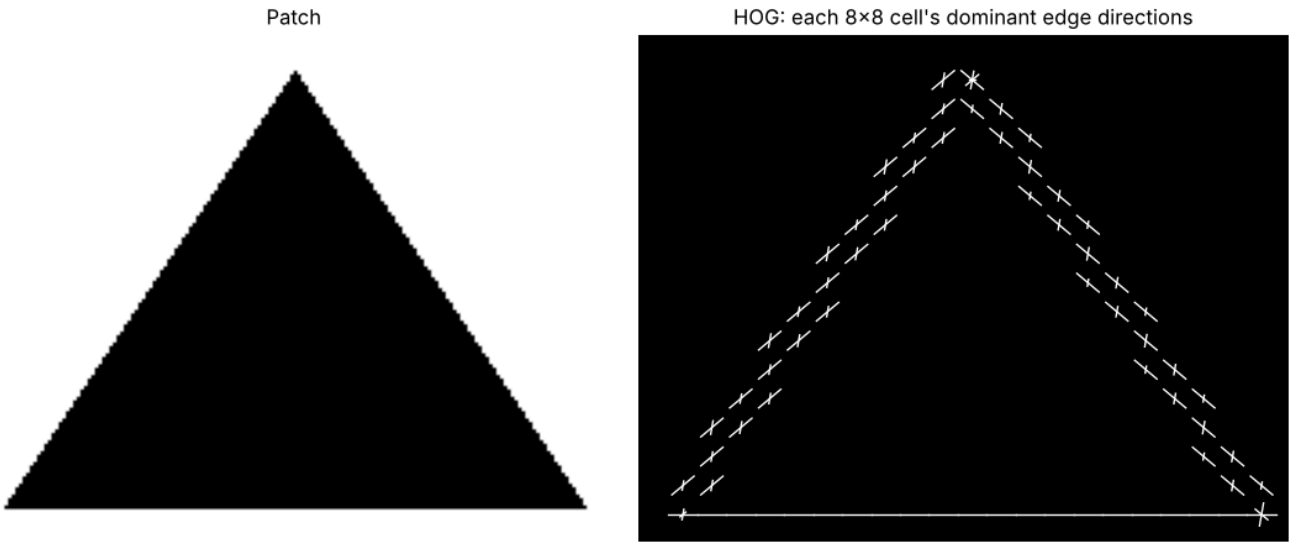
## SURF: Speeded-Up Robust Features

- Introduced in **2006** as a **faster alternative to SIFT**, still scale- and rotation-invariant.
- **Detection:** finds interest points (corners, blobs, junctions) with a **Hessian-matrix-based** measure.
- **Description:** a compact vector of **64 or 128** dimensions.
- **Matching:** compare descriptors across images to establish correspondences.
- It's patented, so it isn't in standard OpenCV builds. **ORB** (`cv2.ORB_create`) is the free, fast alternative used in the demo.

## HOG: Histogram of Oriented Gradients

> Describes a region by the **distribution of edge directions** in it.

**Pipeline:** 1. **Compute gradients** (Sobel x and y → strength and direction). 2. **Divide the image into cells** (e.g. 8×8 pixels). 3. In each cell, **build an orientation histogram**: count gradient directions (usually 9 bins over 0–180°), weighted by strength. 4. **Normalize over blocks** (groups of cells), so lighting changes matter less.

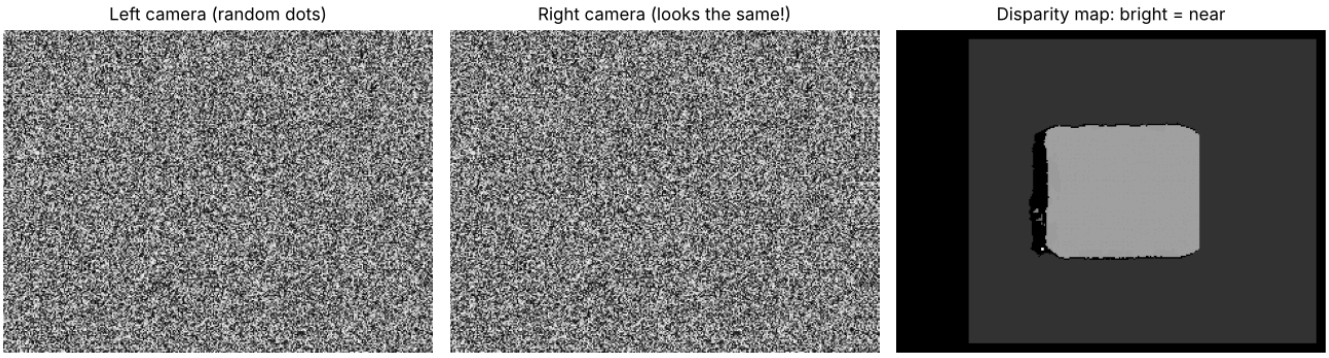



- **Applications: pedestrian detection** (the classic HOG + SVM detector), object classification.
- **Strength:** lightweight and interpretable. **Limit:** sensitive to scale changes.
- Note the connection: HOG is a **histogram** (Day 4) of **gradients** (Sobel, Day 3).

## Stereo vision

> **Two images from slightly different viewpoints** (like your two eyes) → estimate **depth**.

- **Concept:** match corresponding points between the left and right images. The horizontal shift is the **disparity**. **Near objects shift more**, so bigger disparity means closer: depth ∝ (focal length × baseline) / disparity.
- **Applications:** 3D reconstruction, autonomous driving, robotics.
- **Challenge:** finding reliable matches in **texture-less regions** (a blank wall looks the same everywhere).




The demo is a **random-dot stereogram**: the left and right images look like identical noise, but one square is shifted more, and the disparity map reveals it as "closer" (brighter).

## Epipolar geometry

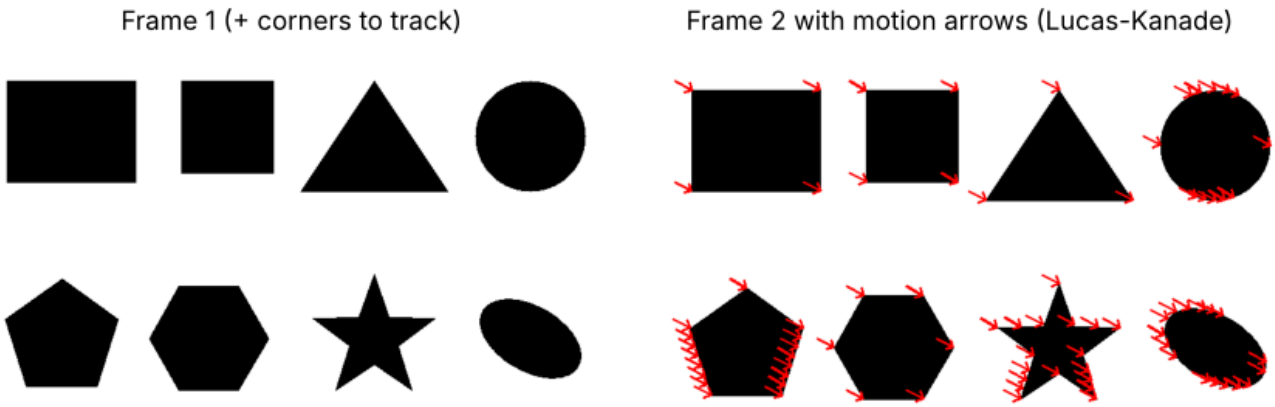
> The **mathematical framework relating two camera views**.

- **Key idea:** for a point in one image, its matching point in the other image **must lie on a specific line**, the **epipolar line**. This turns a 2-D search into a **1-D search**. For aligned ("rectified") stereo cameras, it's simply the **same row**.
- **Tools:** the **Essential matrix** (calibrated cameras, internal parameters known) and the **Fundamental matrix** (uncalibrated).
- **Applications:** stereo matching, structure-from-motion, camera calibration.

## Optical flow

> Estimates the **apparent motion of pixels between consecutive frames**: a motion arrow per point.

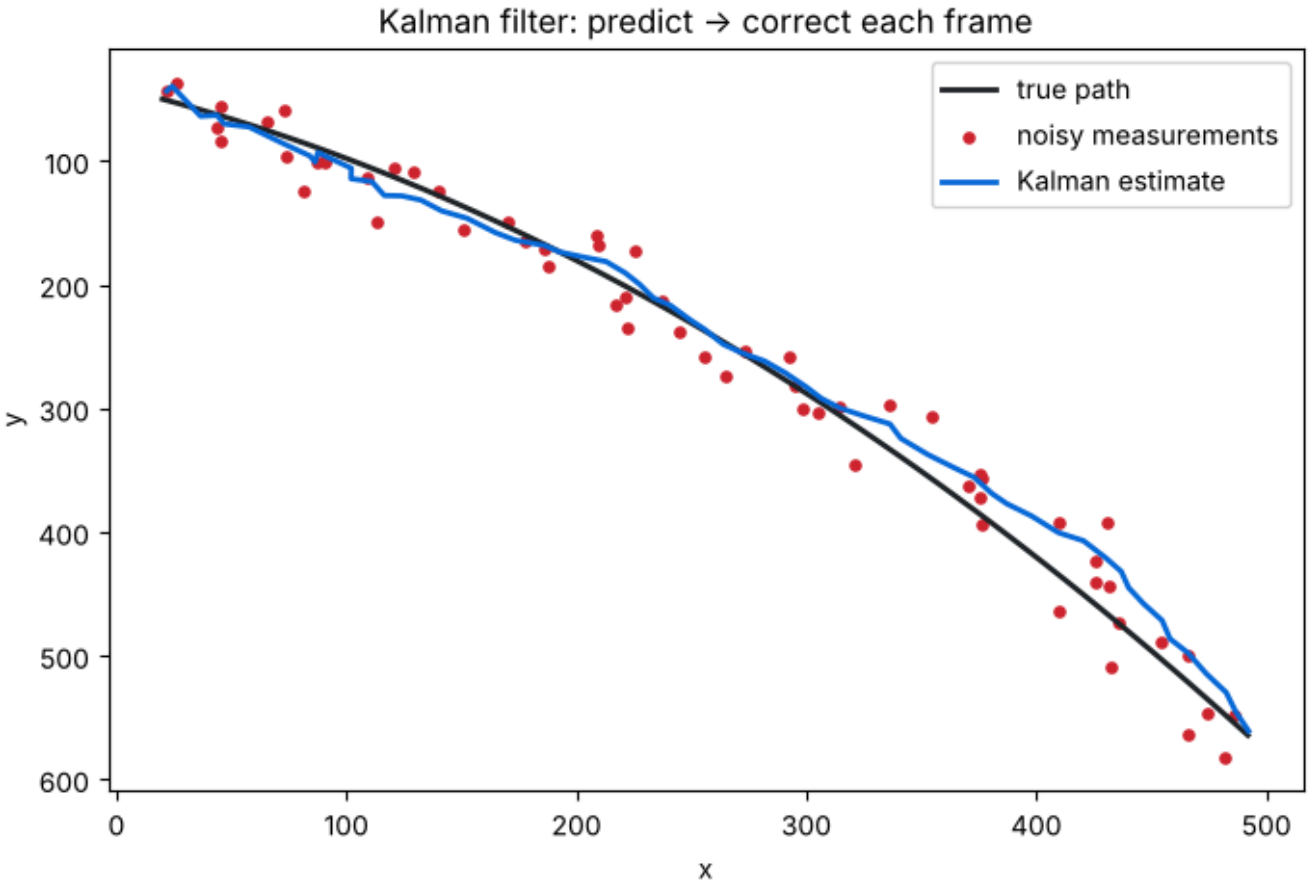
- **How it works:** it assumes a pixel's **brightness stays constant as it moves**, then calculates displacement vectors.
- **Lucas–Kanade:** solves in a **local neighbourhood** around each point. Fast, used for tracking chosen points ( `cv2.calcOpticalFlowPyrLK` ).
- **Horn–Schunck:** solves for the whole image with a **global smoothness constraint**.
- **Applications:** gesture recognition, motion detection, video stabilization.
- **Limitations:** sensitive to **lighting changes** (which break the constant-brightness assumption) and **occlusions** (when something hides the point).




## Kalman filter

> A **recursive** algorithm that **predicts and corrects** the state (position and velocity) of a moving object over time.

- **Prediction step:** estimate the next position from a motion model ("it was moving right at 8 px/frame, so it should be about here").
- **Update step:** correct the prediction using the new (noisy) measurement.
- **Strengths:** handles **noisy sensor data**, smooths trajectories. **Applications:** tracking vehicles, drones and pedestrians in real time.
- Works best for **linear motion with Gaussian noise**. The **Extended Kalman Filter** handles non-linear cases.




In the demo, the raw measurements are off by 18 px on average, while the Kalman estimate is off by about 12 px. It's smoother and closer to the truth.

## Object tracking

> **Continuously locating objects across video frames.**

| Approach | How |
|---|---|
| **Detection-based tracking** | run a detector on each frame (e.g. **YOLO, Faster R-CNN**), then link detections frame to frame |
| **Tracking-by-motion** | follow the object using **optical flow** or **correlation filters** |
| **Hybrid** | detection + prediction, e.g. **SORT / DeepSORT** (which use a Kalman filter to predict) |

- **Challenges:** occlusion, scale changes, multiple objects, cluttered backgrounds.
- **Applications:** surveillance, autonomous driving, sports analytics.

▶ Run `code/keyconcepts_01_corners_and_features.py` and `code/keyconcepts_02_motion_and_depth.py`.

---

## ✅ Key concepts: check yourself

1. Why are corners better landmarks than edges? *(they change in all directions, so they're unique and repeatable; an edge looks the same along its length)*
2. Name 3 corner detectors. *(Harris, Shi-Tomasi, FAST)*
3. The 4 SIFT steps? *(Gaussian scale space → DoG extrema → orientation → 128-D descriptor)*
4. SURF vs SIFT? *(SURF, from 2006, is faster, uses a Hessian-based detector, and has 64/128-D descriptors)*
5. The HOG pipeline? Its classic use? *(gradients → 8×8 cells → orientation histograms → block normalization; pedestrian detection)*
6. What does disparity tell you in stereo vision? *(depth: bigger disparity means closer)*
7. What is the epipolar line? Essential vs Fundamental matrix? *(the line the matching point must lie on; calibrated vs uncalibrated cameras)*
8. Optical flow's key assumption? Lucas–Kanade vs Horn–Schunck? *(brightness constancy; local window vs global smoothness)*
9. The two Kalman steps? *(predict, then update/correct)*
10. Three approaches to object tracking? *(detection-based, tracking-by-motion, hybrid such as SORT/DeepSORT)*

# Week 1 Question Bank: Model Answers

Answers to every question in the course's **CV-Week1-Questions.pdf** (original in `course-files/question-bank/`). Each answer is written the way you'd say it in an eval: short, correct and in plain words. Where it helps, there's a pointer to the note or code that shows it working.

> How to use: cover the answer, say yours out loud, then compare. The "Analytical & Design" ones are the longest. Practise them as 5–8 line answers.

---

## Part 1 · Concept Recall

**1. Grayscale vs colour image as a matrix?**
A grayscale image is a **2-D matrix** `(rows, cols)` with one number per pixel. A colour image is a **3-D matrix** `(rows, cols, 3)` with three numbers per pixel, one per channel (B, G, R in OpenCV). → `notes/day1_images_and_colors.md`

**2. What do 0 and 255 mean in a grayscale image?**
0 = **black** (no light), 255 = **white** (full intensity). Values in between are shades of grey. 255 is the maximum because pixels are stored in 8 bits ($2^8 = 256$ values).

**3. How many channels does an RGB image have?**
**Three**: Red, Green and Blue.

**4. Where is the coordinate origin in CV images?**
At the **top-left corner** `(0, 0)`. x increases to the right and **y increases downward**.

**5. How do you access the pixel at (x = 10, y = 5) in OpenCV?**
`img[5, 10]`: **row (y) first, then column (x)**. For a colour image it returns `[B, G, R]`.

**6. Default colour order in OpenCV?**
**BGR** (Blue, Green, Red), not RGB.

**7. What does HSV stand for?**
**Hue, Saturation, Value.**

**8. What does Hue represent?**
The **type of colour** (red, green, blue…), measured as an **angle on the colour wheel**: 0° = red, 120° = green, 240° = blue. OpenCV stores it halved, 0–179.

**9. What does Saturation indicate?**
How **pure or intense** the colour is: 0 = grey (washed out), maximum = fully vivid.

**10. What does Value indicate?**
The **brightness**: 0 = black, maximum = the brightest version of that colour.

**11. Define a kernel.**
A **small matrix of numbers** (e.g. 3×3 or 5×5), also called a filter, mask or convolution

matrix, that slides over an image. At each position it defines how a pixel and its neighbours are **weighted and combined** into a new output pixel.

**12. Purpose of convolution in image filtering?**
To **extract or change features**: blur (smooth), sharpen, or detect edges and textures. You slide a kernel over the image and at each spot **multiply and sum**. The output is a feature map.

**13. What does a histogram represent?**
The **distribution of pixel intensities**: how many pixels have each value (x-axis 0–255, y-axis = count). It shows brightness and contrast.

**14. What is thresholding used for?**
To convert a grayscale image to **binary** (black and white): pixels above a cut-off become 255 and the rest become 0. This **separates foreground from background**, e.g. text from paper or an object from the background.

**15. Name two basic morphological operations.**
**Dilation** and **erosion**. Opening and closing are combinations of the two.

---

## Part 2 · Application & Explanation

**1. Why is HSV preferred over RGB for colour detection?**
HSV **separates colour (Hue) from brightness (Value)**. When lighting changes (shadow or sunlight), mostly V changes while H stays about the same. So you can detect "red" with a simple Hue range. In RGB all three numbers change with the light, so a fixed colour range breaks easily. → `code/day1_03_color_spaces.py`

**2. An RGB pixel value for pure red?**
`(255, 0, 0)` in RGB. *(In OpenCV's BGR order the same red is stored as `[0, 0, 255]`.)*

**3. What happens when you apply a blur kernel?**
Each pixel is replaced by an **average of its neighbours** (equal weights for a box blur, bell-curve weights for a Gaussian blur). Sharp changes get smoothed out: **noise and fine detail are reduced** and edges become softer. A bigger kernel means more blur. The kernel's numbers sum to 1, so overall brightness stays the same.

**4. How does an edge-detection kernel work?**
Its numbers **sum to 0** and have opposite signs on opposite sides (e.g. Sobel-X `[[-1,0,1],[-2,0,2],[-1,0,1]]`). Over a **flat area** the positives and negatives cancel, giving **0**. Where brightness **changes suddenly** (an edge) they don't cancel, giving a **large value**. So the output lights up exactly on edges. Sobel-X finds vertical edges and Sobel-Y finds horizontal ones. → `code/day3_01_convolution_by_hand.py`

**5. Compare global and adaptive thresholding.**
- **Global** uses **one threshold for the whole image**. It's simple and fast, but fails when lighting is uneven. - **Adaptive** computes a **separate threshold for each small neighbourhood** (local mean or Gaussian-weighted mean minus C). It handles shadows and uneven lighting, but has more parameters (blockSize, C) and can turn flat areas speckled. - **Otsu** is a global method that picks the threshold automatically. → `notes/day4_histograms_and_thresholding.md`

**6. What does a histogram tell you about brightness and contrast?**
- **Where** the pixels sit tells you **brightness**: bunched on the left means dark (underexposed), on the right means bright (overexposed). - **How spread out** they are tells you **contrast**: a narrow hump means low contrast (flat, grey), spread across 0–255 means high contrast. - Two separate humps (**bimodal**) mean a clear object and background, which is ideal for thresholding.

**7. Dilation vs erosion?**
- **Dilation** (local maximum) **expands white regions**: it thickens objects, connects broken parts and fills small gaps. - **Erosion** (local minimum) **shrinks white regions**: it thins objects, removes small specks and separates touching objects. - Both use a structuring element. Its size controls how strong the effect is.

**8. Opening vs closing?**
- **Opening = erosion then dilation**: removes **small white noise** (specks) while keeping large shapes about the same size. - **Closing = dilation then erosion**: fills **small black holes and gaps** inside objects. - Analogy: opening erases specks; closing fills gaps.

**9. Why are structuring elements important?**
They **define the neighbourhood** the operation looks at, so they decide the result. - **Size** sets the strength: a bigger element removes bigger noise but also destroys more detail. - **Shape** (rectangle, ellipse, cross) decides what the edges and corners look like afterwards: square corners, rounded corners, or plus-shaped growth. - Choose it **bigger than the noise but smaller than the detail you want to keep**.

**10. What is a convex hull?**
The **smallest convex polygon that encloses a contour**, like a rubber band snapped around the shape. It simplifies the shape. Comparing area to hull area (**solidity**) reveals dents, e.g. a star ≈ 0.5 while a square ≈ 1. → `notes/day6_contours_and_shapes.md`

**11. How are bounding boxes used for object localization?**
`cv2.boundingRect(contour)` returns `(x, y, w, h)`, the **smallest upright rectangle around the object**. It says **where** the object is in compact form: you can draw it, crop it out, track it between frames, count objects, or measure their size and aspect ratio. Detection models (YOLO, SSD) output exactly this kind of box.

**12. Why is the origin (0, 0) at the top-left?**
Because of how images are **stored and displayed**. Screens and old CRT TVs draw the picture **row by row from the top-left**, and image files and memory store pixels in that same order. So row 0 is the top row. That's also why images are matrices indexed `[row, col]`, and why y grows downward.

**13. How does Matplotlib's RGB order differ from OpenCV's BGR?**
They store the same three channels in **opposite order**: OpenCV `[B, G, R]`, Matplotlib `[R, G, B]`. Show a `cv2.imread` image with `plt.imshow` without converting and **red and blue swap** (orange looks blue). Fix: `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`.

**14. Role of kernels in CNNs vs classical image processing?**
- **Classical:** a **human designs** the kernel numbers for a known job, e.g. Sobel for edges or a box filter for blur. - **CNN:** the kernel numbers are **learned automatically** from data during training. Early layers end up learning edge and colour detectors (much like Sobel), and deeper

layers combine them into textures, parts and whole objects. - The **operation is the same** (slide, multiply, sum). Only who chooses the numbers differs.

**15. How can thresholding be applied in medical imaging?**
Tissues show up at different **intensity ranges** in scans such as X-ray, CT and MRI. Thresholding can **segment** the region of interest, e.g. bone (very bright on CT), a tumour or a lesion. You then clean the mask with morphology and measure its **area or volume** with contours. Otsu or adaptive thresholding cope with varying scan brightness.

---

## Part 3 · Analytical & Design-Oriented

**1. Derive the mathematical expression for 2-D convolution.**
The output at (x, y) is a weighted sum of the input pixels around (x, y), with weights from the kernel K:

$$(I * K)(x, y) = \sum_m \sum_n I(x - m, y - n) \cdot K(m, n)$$

m and n run over the kernel's positions, e.g. −1..1 for 3×3. The minus signs mean the kernel is **flipped**. That's what makes it true convolution. `cv2.filter2D` and CNNs actually compute **correlation**, Σ I(x + m, y + n)·K(m, n), with no flip. For symmetric kernels the two are identical. Output size without padding: **(N − K + 2P)/S + 1**.

**2. Limitations of global thresholding under uneven illumination, and solutions.**
- **Problem:** one cut-off can't fit both the dark and bright parts. With a shadow, the dark side turns entirely black (text lost) or the bright side loses faint text. Otsu doesn't help, because it's still one global value. - **Solutions:** - **Adaptive thresholding**: a local threshold per neighbourhood, Gaussian or mean. - Blur first to avoid speckle. - **Background normalisation**: estimate the background with a large blur or morphological closing, then divide or subtract it before thresholding. - **CLAHE** to even out local contrast. - Split the image into tiles and use Otsu per tile.

→ `code/day4_02_thresholding.py` (uneven-lighting demo)

**3. How does histogram equalization improve contrast?**
A low-contrast image uses only a **narrow range** of intensities (a squashed histogram). Equalization remaps pixel values using the **cumulative histogram (CDF)**, new value = 255 × CDF(old value), so the output intensities are **spread across the full 0–255 range**. Similar dark and bright values get pulled further apart, so details become visible. `cv2.equalizeHist(gray)`. **CLAHE** does it per tile with a clip limit, so it doesn't over-boost noise. → `code/day4_01_histograms.py`

**4. Design a pipeline to detect ripe fruit using HSV.**
1. Capture image → **blur** slightly (reduce noise). 2. **Convert BGR → HSV.** 3. **Threshold on Hue** for the ripe colour (e.g. red: H 0–10 *and* 170–179; ripe banana yellow: about 20–35), with minimum **S** (to ignore grey) and **V** (to ignore dark shadow). Use `cv2.inRange` to get a mask. 4. **Morphology**: opening to remove specks, closing to fill holes. 5. **Find contours**, drop the small ones (area filter), optionally check circularity. 6. Output **bounding boxes / count** of ripe fruit. Compare the ripe-hue mask with an unripe-hue mask (green, H about 40–80) to report ripeness.

Why HSV: ripeness is a **colour (Hue)** property, and HSV keeps it stable under changing light.

**5. How is contour approximation (Douglas–Peucker) used for shape classification?**
`cv2.approxPolyDP(cnt, ε, True)` **simplifies a contour into a polygon** with fewer vertices. Douglas–Peucker keeps a point only if it lies **farther than ε** from the line joining its neighbours. Choose ε relative to the size, e.g. **ε = 0.02 × perimeter**, then **count the vertices**: 3 = triangle, 4 = rectangle/square (check aspect ratio w/h ≈ 1 for a square), 5 = pentagon, many = circle. A large ε over-simplifies (a circle can become a square). Add solidity or circularity to handle stars and ellipses. → `code/day6_02_boxes_hull_shapes.py`

**6. How can morphology clean noisy scanned documents?**
1. Grayscale → **threshold (Otsu or adaptive) with INV**, so the text is white. 2. **Opening** with a small kernel (bigger than the specks, smaller than the strokes) **removes salt noise**. 3. **Closing fills small gaps and holes in the letters**. 4. Invert back to black text on white.

The order and kernel size matter. If letters have pinholes, close first. A kernel that's too big erases thin strokes. → `code/day5_03_mini_project_document_cleaning.py`

**7. Separate overlapping objects using thresholding, morphology and contours.**
1. **Threshold** (Otsu) to get a foreground mask. 2. **Opening** to remove noise. **Erosion** shrinks the objects until the thin "bridges" where they touch disappear, splitting them. 3. A better option: compute the **distance transform** ( `cv2.distanceTransform` ). Each object's centre becomes a peak, so threshold the peaks to get one **marker** per object. 4. Use the markers in **watershed** ( `cv2.watershed` ), or dilate them back, to recover each object's full extent. 5. **findContours** on the separated regions → count, box and measure each one.

**8. RGB vs HSV for detecting traffic lights under varying lighting.**
- **RGB:** a red light at night, at noon, or in glare gives very different R, G and B values. Fixed RGB ranges either miss lights or fire on other bright objects, so performance drops sharply as lighting changes. - **HSV:** the light's **Hue** stays roughly constant (red about 0/179, amber about 15–25, green about 50–90). Brightness changes mostly affect V. So use a Hue range plus **high S and high V** (lights are vivid and bright). This is much more robust. - **Caveats:** glare can wash out saturation, white balance can shift hue, and red needs two Hue ranges because it wraps around. Add **shape (circularity)** and **position** checks via contours.

**9. How does kernel size affect convolution?**
- A **larger kernel** looks at a **bigger neighbourhood**: stronger blur and smoothing, captures **coarser and larger structures**, removes more noise but loses fine detail and blurs edges, **costs more computation** ($K^2$ multiplications per pixel) and **shrinks the output more** without padding. - A **smaller kernel** gives finer detail but is more noise-sensitive. - In CNNs (VGG), stacking small 3×3 kernels reaches a large receptive field with fewer parameters.

**10. How is convolution related to feature extraction in CNNs?**
A CNN layer is **many convolutions in parallel**, each with a **learned kernel**. Each kernel produces a **feature map** showing where its pattern appears. The layers form a hierarchy: - **Layer 1** learns edges and colour blobs (like Sobel). - **Middle layers** combine those into textures and parts. - **Deep layers** respond to whole objects.

Pooling shrinks the maps, and the final layers classify. So convolution is the **feature extractor**, and training finds the kernel weights automatically. → `week-2-3/notebooks/01_…`

**11. How can histograms detect underexposed or overexposed images?**
- **Underexposed:** most pixel counts are at the **low end** (hump near 0) and the mean is low. A

spike at exactly 0 means **clipped shadows**. - **Overexposed:** most counts are at the **high end**, and a spike at 255 means **clipped highlights**.

For an automatic check, compute the mean and the percentage of pixels below about 20 or above about 235 and compare with thresholds. Fixes include **gamma correction** or histogram equalization.

**12. Trade-offs between small and large structuring elements.**
- **Small:** removes only tiny noise, **preserves detail** and thin structures, but may leave larger noise and can't bridge large gaps. - **Large:** removes bigger noise and fills bigger gaps, but **destroys small objects and thin lines**, rounds or merges shapes, and costs more computation.

Rule: **bigger than the noise, smaller than the smallest feature you need**. Shape matters too: an ellipse gives rounder results, a rectangle keeps corners square. You can also use a small kernel with **iterations**.

**13. How can contour features (area, perimeter, centroid) be used for classification?**
They form a numeric **feature vector** per object: - **Area** separates small from large objects (and filters noise). - **Perimeter** and **circularity = $4\pi \cdot A / P^2$** separate round from elongated or jagged shapes. - **Aspect ratio** (w/h), **extent** (area / box area) and **solidity** (area / hull area) separate rectangles, circles and stars. - **Centroid** gives the position, used for tracking or for "which lane or zone" decisions.

Classify with simple **rules** (thresholds) or feed the features to an ML classifier (k-NN, SVM). Example: coins sorted by area, or nuts vs bolts by circularity.

**14. Detect cracks in industrial materials using convolution and morphology.**
1. Grayscale → **blur** (Gaussian) to suppress surface texture noise. 2. Cracks are **thin dark lines**. Use **edge or line convolution**: Sobel, Laplacian, Canny, or a **black-hat** transform (closing − image), which highlights thin dark structures. 3. **Threshold** (Otsu or adaptive) to get a binary crack mask. 4. **Morphology:** closing joins broken crack segments, and opening or an area filter removes small specks. 5. **Contours:** keep long, thin shapes (large length-to-width ratio, low solidity). Measure length and width and flag the part if they exceed tolerances.

**15. How does combining HSV segmentation with contour analysis improve detection accuracy?**
- **HSV segmentation** finds pixels of the **right colour**, robustly under lighting changes. But colour alone gives false positives (anything with the same colour) and a noisy mask. - **Contour analysis** adds **shape and size checks**: area range, circularity, aspect ratio and convexity. It also groups pixels into **distinct objects** with positions.

Together they reject wrong-shaped blobs, remove noise and give per-object boxes and counts. Example: traffic lights must be red **and** round **and** a certain size. That gives far fewer false positives than either check alone.

# Week 1 Practice Quiz

Answer each question **out loud or on paper first**, then click ▸ to check. Questions are in the styles evals usually use: concepts, "what does this code do", "predict the output", and "find the bug".

---

## Part A · Concepts (Day 1)

**A1.** What is Computer Vision? Give 3 applications.

> **Answer:** The field of AI that lets machines interpret and understand visual information (images and video). Applications: self-driving cars, medical imaging (tumour detection), surveillance, augmented reality, traffic monitoring, retail (face recognition, product tracking), creative AI (art generation).

**A2.** What is a pixel? What values can a pixel in an 8-bit grayscale image have?

> **Answer:** The smallest square ("picture element") of a digital image. In an 8-bit grayscale image each pixel is one whole number from 0 (black) to 255 (white), because 8 bits give $2^8$ = 256 values.

**A3.** An image has `img.shape == (720, 1280, 3)`. What are its width, height and number of channels? How many numbers does it store in total?

> **Answer:** Height 720, width 1280, 3 channels. 720 × 1280 × 3 = 2,764,800 numbers.

**A4.** What does "resolution" mean and why does it matter?

> **Answer:** The number of pixels (width × height). More pixels = more detail, but also more memory and slower processing.

**A5.** In which order does OpenCV store colour channels? What goes wrong if you ignore this?

> **Answer:** BGR (Blue, Green, Red). Matplotlib expects RGB, so if you show a cv2 image with plt.imshow without converting, red and blue swap. Fix: `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`.

**A6.** What do H, S and V stand for? What is the range of H in OpenCV and why?

> **Answer:** Hue (which colour), Saturation (how vivid/pure), Value (how bright). OpenCV's H is 0–179: the real range is 0–360°, and halving it lets it fit in a uint8 (max 255).

**A7.** Why is HSV preferred over RGB for colour detection?

> **Answer:** HSV keeps the colour (Hue) separate from the brightness (Value). Lighting changes like shadows mostly change V, so you can detect a colour with a Hue range that works under different lighting. In RGB, lighting changes all three numbers.

**A8.** Grayscale formula used by OpenCV? Why does green get the biggest weight?

> **Answer:** Y = 0.299R + 0.587G + 0.114B. Human eyes are most sensitive to green light.

**A9.** What is a mask?

> **Answer:** A single-channel black/white image of the same size as the original: 255 (white) = keep/selected pixel, 0 (black) = ignore. Made with e.g. `cv2.inRange` and used with `cv2.bitwise_and`.

---

## Part B · Concepts (Day 2)

**B1.** Write the translation matrix that moves an image 40 px right and 25 px up.

> **Answer:** `np.float32([[1, 0, 40], [0, 1, -25]])`. Up is a negative y because y grows downwards.

**B2.** In `cv2.getRotationMatrix2D(center, angle, scale)`, what does each argument do? Which direction is a positive angle?

> **Answer:** center = pivot point (x, y); angle in degrees; scale = resize factor (1 = same size). Positive angle = anti-clockwise.

**B3.** Why do the corners of an image get cut off after rotating by 45°?

> **Answer:** The output size given to warpAffine is the same as the original (cols, rows), but a rotated rectangle needs a bigger box, so the corners fall outside the canvas. Empty areas become black.

**B4.** What is the defining property of an affine transformation? How many points define it?

> **Answer:** Parallel lines stay parallel (and straight lines stay straight). It's defined by 3 point pairs, giving a 2×3 matrix.

**B5.** How is a perspective transformation different from an affine one? Give one real-world use.

> **Answer:** Perspective maps any quadrilateral to any quadrilateral using 4 point pairs and a 3×3 matrix; parallel lines can converge. Used for document scanning (flattening a page photographed at an angle), bird's-eye view of a road, and so on.

**B6.** Which interpolation would you choose to shrink an image? To enlarge it smoothly?

> **Answer:** Shrink: `cv2.INTER_AREA`. Enlarge smoothly: `cv2.INTER_CUBIC` (or INTER_LINEAR, the default, which is faster).

**B7.** What is data augmentation and why is it used?

> **Answer:** Creating modified copies of training images (shifted, rotated, scaled, flipped, warped…) so a model sees more variety. This reduces overfitting and makes the model robust to position, size and angle changes.

**B8.** Translation, scaling and rotation: are they affine?

> **Answer:** Yes. All three (and shear) are special cases of affine transformations.

---

## Part C · Concepts (Day 3)

**C1.** Define kernel, convolution and feature map.

> **Answer:** Kernel: a small matrix of weights (e.g. 3×3). Convolution: sliding the kernel over the image and at each position computing the weighted sum (multiply element-wise, then add). Feature map: the output image of those sums, which highlights where the pattern occurs.

**C2.** Compute the output of this 3×3 patch with the sharpen kernel (centre value only):

```
patch = [[10, 10, 10],        kernel = [[ 0,-1, 0],
         [10, 50, 10],                  [-1, 5,-1],
         [10, 10, 10]]                  [ 0,-1, 0]]
```

> **Answer:** 5×50 − 10 − 10 − 10 − 10 = 250 − 40 = **210**. The bright centre gets even brighter compared with its neighbours: sharpening.

**C3.** Same patch, with the 3×3 box (average) blur kernel?

> **Answer:** (8×10 + 50)/9 = 130/9 ≈ **14.4**. The bright spot gets smoothed away.

**C4.** Kernel sums: what do they tell you? Give the sum for blur, sharpen and Sobel.

> **Answer:** Sum 1 keeps overall brightness (blur = 1, sharpen = 1). Sum 0 sends flat regions to 0, keeping only changes, i.e. edges (Sobel = 0, Laplacian = 0).

**C5.** Difference between average blur and Gaussian blur?

> **Answer:** Average blur weights all neighbours equally. Gaussian blur weights them with a bell curve, so closer pixels count more. Gaussian looks more natural and preserves edges a bit better.

**C6.** Why must kernel sizes like (5, 5) be odd?

> **Answer:** So the kernel has a single centre pixel to place on the pixel being computed.

**C7.** What do `dx=1, dy=0` in `cv2.Sobel` detect? Why is `cv2.CV_64F` used?

> **Answer:** Changes in the x direction, which means **vertical** edges. CV_64F (float) is used because gradients can be negative (bright→dark). uint8 would clip negatives to 0 and lose those edges.

**C8.** List the 4 stages of the Canny edge detector.

> **Answer:** 1) Gaussian noise reduction, 2) gradient computation (Sobel: magnitude + direction), 3) non-maximum suppression (thin edges to 1 px), 4) hysteresis thresholding with low/high thresholds.

**C9.** In `cv2.Canny(img, 100, 200)`, classify pixels with gradient strength 250, 150 and 50.

> **Answer:** 250 → strong edge (kept). 150 → weak, kept only if connected to a strong edge. 50 → discarded.

**C10.** What happens to the Canny result if you lower both thresholds?

> **Answer:** More edges are detected, including noise and texture. Raising them keeps only the strongest outlines.

**C11.** Sobel vs Laplacian?

> **Answer:** Sobel is a first derivative, computed separately in x or y (it has a direction). Laplacian is a second derivative that detects edges in all directions with one kernel, but it is more sensitive to noise.

**C12.** Why is convolution called "the foundation of CNNs"?

> **Answer:** CNN layers are made of convolution kernels. Instead of hand-designing kernels (like Sobel), the network learns the kernel values from data. Early layers end up learning edge-like filters.

**C13.** Output size of a 28×28 image convolved with a 3×3 kernel, stride 1, no padding? With padding 1?

> **Answer:** $(28 - 3 + 0)/1 + 1 = 26 \rightarrow 26\times26$. With $P = 1$: $(28 - 3 + 2)/1 + 1 = 28 \rightarrow 28\times28$ (same size).

**C14.** What is a contour? What do `RETR_EXTERNAL` and `CHAIN_APPROX_SIMPLE` mean?

> **Answer:** A contour is a curve of points along the boundary of a shape. RETR_EXTERNAL returns only the outermost contours. CHAIN_APPROX_SIMPLE compresses straight segments down to their end points to save memory.

**C15.** The edge-based object detector draws boxes around lots of tiny noise. Give 2 fixes.

> **Answer:** Blur before Canny; skip contours with `cv2.contourArea(cnt) < threshold`; dilate the edges to close gaps; tune the Canny thresholds.

**C16.** Name one application of filtering in each of medical imaging, document scanning and autonomous driving.

> **Answer:** Tumour boundary detection; detecting page edges / sharpening text; lane detection.

---

## Part D · Predict the output

**D1.**

```
img = cv2.imread('photo.jpg')     # a 400 × 600 colour photo (height 400)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
print(gray.shape)
```

> **Answer:** `(400, 600)`. Grayscale has no channel dimension.

**D2.**

```
small = cv2.resize(img, None, fx=0.5, fy=0.25)
print(small.shape)               # img is (400, 600, 3)
```

> **Answer:** `(100, 300, 3)`. Width 600×0.5 = 300, height 400×0.25 = 100.

**D3.**

```
a = np.array([200], dtype=np.uint8)
print(a + a)
```

> **Answer:** `[144]`. 400 wraps around: 400 − 256 = 144. ( `cv2.add` would give 255.)

**D4.**

```
edges = cv2.Canny(img, 100, 200)
print(edges.shape, np.unique(edges))      # img is (400, 600, 3)
```

> **Answer:** `(400, 600) [ 0 255]`, a single-channel image containing only 0 and 255 (assuming some edges were found).

**D5.**

```
img = np.zeros((100, 200, 3), np.uint8)
img[:, :100] = (0, 0, 255)
print(img[50, 150], img[50, 50])
```

> **Answer:** `[0 0 0] [ 0 0 255]`. The left half (columns 0–99) is red in BGR. Pixel (row 50, col 150) is on the right, so it's black.

**D6.**

```
hsv = cv2.cvtColor(np.uint8([[[0, 255, 0]]]), cv2.COLOR_BGR2HSV)
print(hsv)
```

> **Answer:** `[[[ 60 255 255]]]`. Pure green has hue 120°, which OpenCV halves to 60.

---

## Part E · Find the bug

**E1.**

```
img = cv2.imread('sample.jpg')
plt.imshow(img)          # colours look wrong
```

> **Answer:** BGR shown as RGB. Use `plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))`.

**E2.**

```
M = np.float32([[1, 0, 100], [0, 1, 50]])
translated = cv2.warpAffine(img, M, (rows, cols))
```

> **Answer:** The size must be (width, height) = `(cols, rows)`. Also `rows, cols` must be defined first: `rows, cols = img.shape[:2]`.

**E3.**

```
blur = cv2.GaussianBlur(img, (4, 4), 0)
```

> **Answer:** The kernel size must be odd, e.g. (3, 3) or (5, 5). An even size raises an error.

**E4.**

```
sobel = cv2.Sobel(gray, cv2.CV_8U, 1, 0)
```

> **Answer:** CV_8U can't hold negative gradients, so bright→dark edges are lost. Use `cv2.CV_64F` and then `cv2.convertScaleAbs`.

**E5.**

```
img = cv2.imread('sampel.jpg')
print(img.shape)          # AttributeError: 'NoneType' object has no attribute
'shape'
```

> **Answer:** The file name is misspelled or the path is wrong. `cv2.imread` doesn't raise an error; it returns `None`. Always check `if img is None`.

**E6.**

```
M = cv2.getPerspectiveTransform(pts1, pts2)
out = cv2.warpAffine(img, M, (300, 300))
```

> **Answer:** A perspective matrix is 3×3 and must be applied with `cv2.warpPerspective`.

**E7.**

```
edges = cv2.Canny(img, 100, 200)
plt.imshow(cv2.cvtColor(edges, cv2.COLOR_BGR2RGB))
```

> **Answer:** Canny returns a single-channel image, so a BGR→RGB conversion makes no sense: there are no colour channels to swap. In OpenCV 4.x it raises an "Invalid number of channels" error; newer versions quietly copy the grey into 3 channels, which is pointless. Use `plt.imshow(edges, cmap='gray')`.

---

## Part F · Day 4: Histograms & Thresholding

**F1.** What does an image histogram show, and what is on each axis?

> **Answer:** The distribution of pixel intensities: how many pixels have each brightness. x-axis = intensity (0–255), y-axis = number of pixels (frequency).

**F2.** A histogram has almost all its pixels between 200 and 255. Describe the image. What if they're all between 100 and 140?

> **Answer:** 200–255: very bright / overexposed. 100–140: low contrast (flat, greyish). Histogram equalization would stretch it out.

**F3.** Explain each argument of `cv2.calcHist([img], [0], None, [256], [0, 256])`.

> **Answer:** List of images; channel index (0, the only channel of a grayscale image); mask (None = the whole image); number of bins (256); value range [0, 256) (the end is excluded).

**F4.** Global vs adaptive thresholding: what's the difference, and when should you use adaptive?

> **Answer:** Global uses one threshold for the whole image. Adaptive computes a separate threshold for each small neighbourhood (its mean or Gaussian-weighted mean minus C). Use adaptive when the illumination varies, e.g. shadows across a page.

**F5.** What does Otsu's method do, and what does it minimise? When does it work best?

> **Answer:** It automatically picks the optimal global threshold by minimising intra-class (within-group) variance, i.e. making the dark and bright groups as uniform as possible. It works best when the histogram is bimodal (two humps: object and background).

**F6.** In `cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)`, why is the threshold 0? What is the first return value?

> **Answer:** With THRESH_OTSU the given threshold is ignored, because Otsu computes it. The first return value is the threshold Otsu chose.

**F7.** What are `blockSize` and `C` in `cv2.adaptiveThreshold(img, 255, cv2.ADAPTIVE_THRESH_MEAN_C, cv2.THRESH_BINARY, 11, 2)`? Mean vs Gaussian method?

> **Answer:** blockSize = 11 is the neighbourhood size (it must be odd and greater than 1). C = 2 is subtracted from the local mean to give the threshold. MEAN_C uses the plain average of the block; GAUSSIAN_C uses a Gaussian-weighted average (nearer pixels count more).

**F8.** Give 3 applications of thresholding.

> **Answer:** Document scanning / binarization, medical imaging (tumour segmentation), industrial inspection (defect detection), object detection (separating foreground from background).

**F9.** Predict: `a = np.array([[50, 127, 128, 200]], np.uint8)`. What is `cv2.threshold(a, 127, 255, cv2.THRESH_BINARY)[1]`? And with `THRESH_TRUNC`?

> **Answer:** BINARY: `[[0, 0, 255, 255]]`. 127 is *not* greater than 127, so it becomes 0. TRUNC: `[[50, 127, 127, 127]]`: values above 127 are capped at 127.

**F10.** Predict: an image that is 100 everywhere goes through `adaptiveThreshold(..., MEAN_C, THRESH_BINARY, 11, 2)`. What comes out?

> **Answer:** All white (255). Each pixel (100) is greater than its local mean minus C (100 − 2 = 98). This is why flat, noisy regions come out speckled: tiny noise decides the result.

**F11.** Predict the histogram (4 bins over 0–256) of `[[0,0,64,64],[0,128,128,64],[255,255,128,64],[255,255,255,128]]`.

> **Answer:** Bins [0–63], [64–127], [128–191], [192–255] → `[3, 4, 4, 5]` (total 16 pixels).

---

## Part G · Day 5: Morphological operations

**G1.** Define dilation and erosion. What does each do to white regions?

> **Answer:** Dilation: a pixel becomes white if ANY pixel under the structuring element is white (local max), so white regions expand. Erosion: a pixel stays white only if ALL pixels under the element are white (local min), so white regions shrink.

**G2.** Define opening and closing (the order of operations), and what each is used for.

> **Answer:** Opening = erosion followed by dilation: it removes small white noise while keeping the shape of large objects. Closing = dilation followed by erosion: it fills small holes and gaps inside objects.

**G3.** What is a structuring element? Name 3 shapes and say what its size controls.

> **Answer:** A small binary matrix used to probe the image. Shapes: rectangle, ellipse, cross (`cv2.getStructuringElement`). The size controls how strong the effect is: a bigger element removes bigger specks but also damages more detail.

**G4.** A binary scan has white text with small black holes inside the letters. Which operation fixes it?

> **Answer:** Closing.

**G5.** A binary image has white objects plus scattered white specks of noise. Which operation fixes it?

> **Answer:** Opening.

**G6.** Predict: a 5×5 black image with one white pixel in the centre. How many white pixels after a 3×3 dilation? After a 3×3 erosion?

> **Answer:** Dilation: 9 (a 3×3 white square). Erosion: 0 (the single pixel doesn't have all of its neighbours white).

**G7.** You have black text on white paper and dilate it. Do the letters get thicker or thinner? Why?

> **Answer:** Thinner! Dilation grows WHITE regions, and here the white is the paper. Invert first (THRESH_BINARY_INV or bitwise_not) so the text is white.

**G8.** Why did the lecture's dilated and eroded photos barely change?

> **Answer:** They were grayscale photos. On grayscale, dilation and erosion are a local max and min, which just slightly brighten or darken and blur the image. The effect is clearest on binary images with small details.

**G9.** What do `iterations=3` and `MORPH_GRADIENT` do?

> **Answer:** iterations=3 repeats the operation 3 times (a stronger effect). MORPH_GRADIENT = dilation − erosion, which gives the outline of the shapes.

**G10.** Name 3 applications of morphology.

> **Answer:** Document cleaning (remove noise, fill gaps), medical imaging (highlight cells and structures), industrial inspection (defects), and cleaning shapes before contour analysis.

---

## Part H · Day 6: Contours & shape analysis

**H1.** What is a contour? Name 4 contour descriptors.

> **Answer:** A curve joining continuous points along the boundary of a shape with the same intensity. Descriptors: area, perimeter, centroid, bounding box, convex hull (also aspect ratio, solidity, circularity).

**H2.** Bounding box vs convex hull: definition and use of each.

> **Answer:** Bounding box: the smallest (upright) rectangle enclosing the contour, used for object localization. Convex hull: the smallest convex polygon enclosing the contour (a rubber band around it), used for shape simplification and finding dents (convexity defects).

**H3.** How does the lecture classify shapes? What is the 0.02 in `approxPolyDP`?

> **Answer:** Approximate the contour to a polygon with `cv2.approxPolyDP(cnt, 0.02*cv2.arcLength(cnt, True), True)` and count the vertices: 3 = triangle, 4 = rectangle, more than 6 = circle, otherwise polygon. 0.02 sets epsilon to 2% of the perimeter, the maximum distance the simplified outline may stray from the real one. A bigger epsilon gives fewer corners.

**H4.** Dark shapes on white paper: `threshold(img, 127, 255, THRESH_BINARY)` then `findContours(..., RETR_EXTERNAL, ...)` returns 1 contour. Why? How do you fix it?

> **Answer:** findContours looks for WHITE objects. THRESH_BINARY makes the paper white and the shapes black, so the only external contour is the page. Use THRESH_BINARY_INV (optionally with Otsu).

**H5.** `RETR_EXTERNAL` vs `RETR_TREE`? `CHAIN_APPROX_SIMPLE` vs `CHAIN_APPROX_NONE`?

> **Answer:** EXTERNAL: outermost contours only. TREE: all contours plus the full parent/child hierarchy. SIMPLE: stores only the end points of straight segments. NONE: stores every boundary point.

**H6.** How do you compute a contour's centroid?

> **Answer:** `M = cv2.moments(cnt)`; `cx = M['m10']/M['m00']`, `cy = M['m01']/M['m00']` (m00 is the area; check it isn't 0).

**H7.** The lecture's rule labels a 5-pointed star as "Circle". Why? How could you detect a star instead?

> **Answer:** The star approximates to 10 vertices, and more than 6 counts as "Circle". Use solidity = area / convex hull area: a star is about 0.5, while convex shapes are about 1.

**H8.** Predict: a white filled rectangle drawn with `cv2.rectangle(img, (10, 20), (50, 40), 255, -1)` on black. What does `cv2.boundingRect` return for its contour? How many points does `CHAIN_APPROX_SIMPLE` store?

> **Answer:** `(10, 20, 41, 21)`. The drawn rectangle includes both corner pixels, so the width is 50 − 10 + 1 = 41 and the height 21. 4 points (one per corner).

**H9.** Predict: a white ring (a filled circle with a smaller black circle cut out). How many contours with `RETR_EXTERNAL`? With `RETR_TREE`?

> **Answer:** EXTERNAL: 1 (just the outer edge). TREE: 2 (outer edge + inner hole, with the hole as a child of the outer contour).

**H10.** Why does `drawContours(img, [hull], 0, ...)` put `hull` in square brackets?

> **Answer:** drawContours expects a LIST of contours. `[hull]` is a list with one contour, and index 0 draws that one.

---

## Part I · Key concepts (advanced topics)

**I1.** Why are corners better features to track than edges?

> **Answer:** At a corner the intensity changes in every direction, so it's uniquely localizable and repeatable. Along an edge, sliding the window looks the same, so the position is ambiguous.

**I2.** Name 3 corner detectors and 3 uses of corners.

> **Answer:** Harris, Shi-Tomasi, FAST. Uses: image matching, tracking, panorama stitching.

**I3.** List the SIFT pipeline. What is it robust to? What is its main limitation?

> **Answer:** Gaussian scale space → Difference-of-Gaussians extrema → orientation assignment → 128-dimensional descriptor. Robust to scale and rotation; computationally heavy.

**I4.** What is SURF and how does it differ from SIFT?

> **Answer:** Speeded-Up Robust Features (2006): a faster alternative to SIFT, also scale- and rotation-invariant. It detects interest points with a Hessian-matrix-based measure and describes them with 64- or 128-dimensional vectors.

**I5.** Describe the HOG pipeline and its classic application.

> **Answer:** Compute gradients → divide into cells (e.g. 8×8) → build an orientation histogram per cell → normalize over blocks. Classic use: pedestrian detection. Lightweight and interpretable, but sensitive to scale.

**I6.** What is stereo vision? What is its main challenge?

> **Answer:** Using two images from slightly different viewpoints to estimate depth by matching corresponding points (closer objects have bigger disparity). The challenge is reliable matching in texture-less regions.

**I7.** What does epipolar geometry say? Essential vs Fundamental matrix?

> **Answer:** A point's match in the other image must lie on the epipolar line, which reduces the search to 1-D. Essential matrix: calibrated cameras. Fundamental matrix: uncalibrated cameras.

**I8.** What is optical flow? What does it assume? Compare Lucas–Kanade and Horn–Schunck.

> **Answer:** It estimates the apparent motion of pixels between consecutive frames, assuming brightness constancy. Lucas–Kanade works in a local neighbourhood; Horn–Schunck uses a global smoothness constraint. It's sensitive to lighting changes and occlusions.

**I9.** Describe the two steps of a Kalman filter. When does it work best?

> **Answer:** Predict (estimate the next state from a motion model) and update (correct it with the measurement). It works best for linear motion with Gaussian noise; the Extended Kalman Filter handles non-linear motion.

**I10.** Name 3 object-tracking approaches with an example of each, and 2 challenges.

> **Answer:** Detection-based (YOLO / Faster R-CNN per frame + linking), tracking-by-motion (optical flow, correlation filters), hybrid (SORT, DeepSORT). Challenges: occlusion, scale change, multiple objects, cluttered backgrounds.

**I11.** Write the 2-D convolution formula and name its parts.

> **Answer:** $(I * K)(x, y) = \Sigma_m \Sigma_n I(x - m, y - n) \cdot K(m, n)$. I = image, K = kernel, (x, y) = output pixel, and m, n run over the kernel.

**I12.** Which kernel highlights vertical edges: Sobel X or Sobel Y? Write it.

> **Answer:** Sobel X: `[[-1,0,1],[-2,0,2],[-1,0,1]]`. It responds to left-to-right changes, which form vertical edges.

# CV Week 1 · Cheat Sheet

## Images = numbers

- Image = grid of **pixels**. `uint8` → values **0 (black) – 255 (white)**.
- `img.shape = (height, width, channels)` = `(rows, cols, 3)`. Grayscale: `(rows, cols)`.
- Pixel access: **`img[y, x]`** (row first). Origin (0,0) = **top-left**, y grows **down**.
- OpenCV size arguments are **(width, height)** = `(cols, rows)`.
- `cv2.imread` returns **`None`** if the path is wrong (no error!).
- uint8 overflow: NumPy wraps (250+10 = 4); `cv2.add` saturates (= 255).

## Colour

| | |
|---|---|
| OpenCV order | **BGR**. Matplotlib wants RGB → `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` |
| Gray | `cv2.COLOR_BGR2GRAY`; Y = 0.299R + 0.587G + 0.114B |
| HSV | `cv2.COLOR_BGR2HSV`. **H** = which colour (**0–179**), **S** = vividness, **V** = brightness (0–255) |
| Why HSV | colour is separate from brightness → robust colour detection |
| Mask | `cv2.inRange(hsv, low, high)` → 255 inside the range; `cv2.bitwise_and(img, img, mask=m)` |
| Hues | red 0/179 · orange ~15 · yellow ~30 · green ~60 · blue ~120 |

## Transformations (Day 2) — `rows, cols = img.shape[:2]`

| Task | Code | Notes |
|---|---|---|
| Translate | `M = np.float32([[1,0,tx],[0,1,ty]])`; `cv2.warpAffine(img, M, (cols, rows))` | +tx right, +ty **down** |
| Scale | `cv2.resize(img, None, fx=.5, fy=.5, interpolation=cv2.INTER_LINEAR)` | shrink: `INTER_AREA`; enlarge: `INTER_CUBIC` |
| Rotate | `M = cv2.getRotationMatrix2D((cols/2, rows/2), angle, scale)` + `warpAffine` | **+angle = anti-clockwise**; corners get cut |
| Affine | `cv2.getAffineTransform(pts1, pts2)` + `warpAffine` | **3 points**, 2×3 matrix, **parallel lines stay parallel** |
| Perspective | `cv2.getPerspectiveTransform(pts1, pts2)` + **`warpPerspective`** | **4 points**, 3×3 matrix, parallel lines can meet → **document scanning** |
| Flip / 90° | `cv2.flip(img, 1)` · `cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)` | |

**Augmentation** = making extra training images by transforming them → more variety, less overfitting.

## Convolution (Day 3)

- **Kernel** = small matrix. **Convolution** = slide it over the image, **multiply element-wise and sum** at each spot. Output = **feature map**. It's the **foundation of CNNs** (which learn their own kernels).
- `cv2.filter2D(img, -1, kernel)` (-1 = same depth as the input). Technically correlation (no kernel flip).
- **Kernel sum = 1** → brightness kept (blur, sharpen). **Sum = 0** → edge detector (flat → 0).
- Output size: **(N − K + 2P)/S + 1**.
- Kernel sizes must be **odd**. Bigger kernel = stronger effect.

| Filter | Code / kernel |
|---|---|
| Average blur | `cv2.blur(img, (5,5))`, all weights equal |
| Gaussian blur | `cv2.GaussianBlur(img, (5,5), 0)`, closer = heavier weight |
| Median | `cv2.medianBlur(img, 5)`, salt & pepper noise |
| Sharpen | `[[0,-1,0],[-1,5,-1],[0,-1,0]]` |
| Sobel | `cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=5)` → dx=1: **vertical** edges; dy=1: horizontal. `[[-1,0,1],[-2,0,2],[-1,0,1]]` |
| Laplacian | `cv2.Laplacian(gray, cv2.CV_64F)`, 2nd derivative, all directions, noise-sensitive. `[[0,1,0],[1,-4,1],[0,1,0]]` |
| Display float | `cv2.convertScaleAbs(result)` → abs + uint8 |

`CV_64F` because gradients can be **negative**; uint8 would lose them.

## Canny — `cv2.Canny(img, low, high)`

1. Gaussian blur (noise) → 2. Sobel gradient (strength + direction) → 3. **Non-max suppression** (1-px thin) → 4. **Hysteresis**: **> high** = strong edge · **< low** = discard · **between** = keep **only if connected to a strong edge**.
2. Output: 1-channel, values **0/255** only. Lower thresholds → more edges/noise. high ≈ 2–3 × low.

## Contours (object detector)

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(cv2.GaussianBlur(gray, (5,5), 0), 50, 150)
contours, hierarchy = cv2.findContours(edges, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
for cnt in contours:
    if cv2.contourArea(cnt) < 500: continue          # skip noise
    x, y, w, h = cv2.boundingRect(cnt)
    cv2.rectangle(img, (x, y), (x+w, y+h), (0,255,0), 2)
    cv2.putText(img, "Obj", (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0,255,0), 1)
```

`RETR_EXTERNAL` = outer outlines only · `CHAIN_APPROX_SIMPLE` = keep corner points only · `approxPolyDP` corner count → shape name.

## Histograms (Day 4) — `img = cv2.imread('x.jpg', 0)` loads grayscale

- Histogram = **count of pixels per intensity** (x: 0–255, y: count). Ignores *where* pixels are.
- `plt.hist(img.ravel(), 256, [0, 256])` · `cv2.calcHist([img], [0], None, [256], [0, 256])` (images, channel, mask, bins, range).
- Hump left = **dark**, right = **bright**, narrow = **low contrast**, two humps = **bimodal** (good for thresholding).
- `cv2.equalizeHist(img)` spreads values out → more contrast.

## Thresholding (Day 4)

| | Code | Notes |
|---|---|---|
| Global | `T, out = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)` | > T → 255, else 0 (127 → 0!) · returns (T, image) |
| Adaptive | `cv2.adaptiveThreshold(img, 255, cv2.ADAPTIVE_THRESH_MEAN_C, cv2.THRESH_BINARY, 11, 2)` | T per neighbourhood = mean (or Gaussian) − C · blockSize **odd** · returns image only · **uneven lighting** |
| Otsu | `T, out = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)` | picks T automatically · **minimises intra-class variance** · best for bimodal |
| - Types: `BINARY`, `BINARY_INV` (dark objects → white), `TRUNC` (cap at T), `TOZERO`, `TOZERO_INV`. | | |
| - Blur before thresholding = less speckle. Uses: document binarization, tumour segmentation, defect detection. | | |

## Morphology (Day 5) — works on WHITE = object (invert dark-on-light first)

| Op | Code | Effect |
|---|---|---|
| Kernel | `np.ones((5,5), np.uint8)` · `cv2.getStructuringElement(cv2.MORPH_RECT/ELLIPSE/CROSS, (5,5))` | shape + size of the probe |
| Dilation | `cv2.dilate(img, k, iterations=1)` | white grows (local **max**) · joins broken parts |
| Erosion | `cv2.erode(img, k, iterations=1)` | white shrinks (local **min**) · removes small noise |
| Opening | `cv2.morphologyEx(img, cv2.MORPH_OPEN, k)` | **erode → dilate** · removes white specks |
| Closing | `cv2.morphologyEx(img, cv2.MORPH_CLOSE, k)` | **dilate → erode** · fills small holes |
| Gradient / top-hat / black-hat | `MORPH_GRADIENT` / `MORPH_TOPHAT` / `MORPH_BLACKHAT` | outline / removed specks / filled holes |
| - Kernel: **bigger than the noise, smaller than the detail you keep.** Marker = dilate, eraser = erode. | | |

## Contours & shapes (Day 6)

```
_, th = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY_INV)        # dark shapes →
WHITE
contours, hierarchy = cv2.findContours(th, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
cv2.drawContours(img_bgr, contours, -1, (0,255,0), 2)               # -1 = all;
thickness -1 = fill
x, y, w, h = cv2.boundingRect(cnt)                                  # upright box
(localization)
hull = cv2.convexHull(cnt); cv2.drawContours(img_bgr, [hull], 0, (0,0,255), 2)   #
rubber band
approx = cv2.approxPolyDP(cnt, 0.02 * cv2.arcLength(cnt, True), True)            #
corners: 3 tri, 4 rect, >6 circle
M = cv2.moments(cnt); cx, cy = M['m10']/M['m00'], M['m01']/M['m00']             #
centroid
```

- `contourArea`, `arcLength(cnt, True)` = perimeter · `minAreaRect` (rotated box) · `minEnclosingCircle`.
- Solidity = area / hull area (star ≈ 0.5) · aspect ratio w/h (square ≈ 1) · circularity $4\pi A/P^2$ (circle ≈ 1).
- `RETR_EXTERNAL` outer only · `RETR_TREE` all + hierarchy `[next, prev, child, parent]` · `CHAIN_APPROX_SIMPLE` = corner points only.
- Uses: object detection/tracking, industrial inspection, cell shape analysis.

## Key concepts (advanced, know what + why)

| Topic | One-liner |
|---|---|
| Corners | intensity changes in **all** directions → unique landmarks · **Harris, Shi-Tomasi, FAST** · matching, tracking, panoramas |
| SIFT | scale space → **DoG** extrema → orientation → **128-D** descriptor · scale/rotation invariant · heavy |
| SURF | 2006, **faster SIFT** · Hessian-based detector · 64/128-D descriptor |
| HOG | gradients → **8×8 cells** → orientation histograms → block normalisation · **pedestrian detection** · scale-sensitive |
| Stereo | two views → match points → **disparity** (bigger = closer) → depth · hard in texture-less areas |
| Epipolar | match lies on the **epipolar line** · **Essential** (calibrated) / **Fundamental** (uncalibrated) matrix |
| Optical flow | pixel motion between frames · assumes **brightness constancy** · **Lucas–Kanade** (local) / **Horn–Schunck** (global smooth) |
| Kalman | **predict → update/correct** · noisy data, smooth tracks · linear + Gaussian (EKF = non-linear) |
| Tracking | detection-based (YOLO, Faster R-CNN) · by motion (optical flow) · hybrid (**SORT / DeepSORT**) · occlusion is hard |
| Convolution | $(I*K)(x,y) = \Sigma_m \Sigma_n I(x-m, y-n)\cdot K(m,n)$ |

## Applications

CV: self-driving cars, medical imaging, surveillance, AR, traffic monitoring, retail (faces, products), creative AI. Filtering: tumour boundaries, document scanning, lane detection.

## Display gotchas

- `plt.imshow(gray, cmap='gray')` for 1-channel images (otherwise you get fake colours).
- Colour from cv2 → convert BGR→RGB first. `cv2.imshow` needs no conversion.